BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION Sisteme informatice avansate

DISSERTATION THESIS

A Reliable and Resilient Closed-Loop Payment System with Modular Offline Support for Festival Environments

Supervisor
Christian Sacarea, Vicerector

Author
Tudor Esan

2026

ABSTRACT

This thesis presents the design and implementation of a cloud-based, closed-loop payment system for music festivals and large outdoor events, featuring a modular offline extension for degraded connectivity scenarios. While festival environments are known to suffer from network instability due to mass gatherings, we argue that a fully offline-first architecture introduces unnecessary complexity for the majority of use cases. Instead, we propose an online-default system with real-time transaction processing and centralized wallet management, augmented by a lightweight offline module that can be easily deployed when network conditions require it. The offline extension leverages local balance snapshots, transaction queuing, and batch synchronization to maintain payment availability during connectivity loss, while keeping the core architecture simple and scalable. We explore the challenges that arise at the boundary between online and offline operation particularly regarding balance consistency, double-spend prevention during offline windows, and conflict reconciliation upon reconnection. Also we will be evaluating strategies for minimizing their impact. The proposed architecture is assessed against the specific requirements of festival environments, demonstrating that treating offline capability as a modular add-on rather than a foundational design principle results in a more practical and maintainable system.

Contents

1 Introduction **1**

1.1 Problem Statement . . . 3

1.2 Objectives . . . 4

1.3 Related Work . . . 5

2 Background and Related Work **7**

2.1 Closed-Loop Payment Systems . . . 7

2.1.1 Transit Card Systems . . . 9

2.1.2 Festival Payment Systems . . . 12

2.1.3 EMV and Offline Authorization . . . 14

2.2 Offline-First Architecture Patterns . . . 15

2.2.1 Local-First Software . . . 16

2.2.2 Conflict-Free Replicated Data Types (CRDTs) . . . 17

2.2.3 Event Sourcing and CQRS . . . 18

2.2.4 Edge Computing . . . 18

2.3 Conflict Resolution in Distributed Systems . . . 19

2.3.1 The CAP Theorem . . . 19

2.3.2 Eventual Consistency . . . 20

2.3.3 Transaction Ordering and Why Timestamps Are Enough . . . 21

3 System Architecture **23**

3.1 High-level overview . . . 23

3.1.1 The hybrid reconciliation model . . . 24

3.2 Balance reconciliation model . . . 25

3.2.1 State on each side . . . 25

3.2.2 Invariants . . . 26

3.2.3 The four operations . . . 27

3.2.4 Worked example . . . 27

3.2.5 Lost transactions . . . 28

3.2.6 Comparison with alternatives . . . 28

3.3 Lessons from related systems and patterns . . . 29

3.3.1 What we learned from existing systems 29
3.3.2 What we took from the architectural patterns 30
3.4 NFC wristbands and offline support 32
3.4.1 How a payment works 32
3.4.2 Offline operation 33
3.4.3 Top-ups: the one thing that needs the network 33
3.4.4 Security of the wristband balance 33
3.5 The mobile application 34
3.5.1 Attendee view 34
3.5.2 Vendor POS mode 34
3.5.3 Device security checks 35
3.6 Admin dashboard 35
3.7 Backend server 36
3.7.1 Why a single service, not microservices 36
3.7.2 Multi-tenancy 37
3.7.3 Horizontal scalability 37
3.8 Database: PostgreSQL 38
3.8.1 Why PostgreSQL 38
3.8.2 Why a single database is enough 38

4 Implementation **40**
4.1 Implementation overview 40
4.2 Database schema for the hybrid model 41
4.3 The reconciliation service and sync endpoint 42
4.4 Chip data layout and HMAC 44
4.5 Online debit path 45
4.6 Offline debit path and the queue 46
4.7 Topup path 47

5 Security Considerations **48**
5.1 Authentication and Authorization 48
5.1.1 Roles 49
5.1.2 The ScopeService 49
5.2 Wallet Integrity and Double-Spend Prevention 50
5.2.1 Online Mode 50
5.2.2 Offline Mode 51
5.2.3 NFC chip authoritativeness for offline debits 51
5.3 Offline Trust Model 54
5.4 Stripe and Payment Security 55
5.5 Data Protection 55

6 Discussion **57**
6.1 Design Trade-offs 57
6.2 Comparison with Existing Solutions 57
6.3 Limitations 57
6.4 Further Improvements 57

7 Conclusions **58**

Bibliography **59**

Chapter 1

Introduction

Music festivals and large outdoor events have become a massive part of modern culture. Every summer, millions of people gather in open fields, parks, and temporary venues to enjoy live performances and food. From Untold in Romania to Tomorrowland in Belgium, these events keep growing in size and complexity. But behind all the excitement, there is a very practical problem that organizers have to deal with: how do you let tens of thousands of people buy food, drinks, and merchandise in a place where the internet barely works?

The traditional approach was straightforward: people brought cash, stood in line, and paid at the counter. It was slow, it was messy with always having change and there was always the risk of theft or losing your wallet in the crowd. Over the past decade, many festivals have moved towards cashless payment systems, typically based on RFID wristbands or prepaid cards. The idea is simple: you load money onto your wristband before or during the event, and then you just tap it at a vendor terminal to pay. Industry data from cashless technology providers suggests that RFID systems reduce transaction times by up to 80 percent compared to cash and that festival organizers see average revenue increases of around 22 percent after going cashless [Tap24]. These figures are consistent with the academic evidence: a recent meta-analysis across 71 studies found a statistically significant, small positive effect of cashless payments on consumer spending ($g = 0.135$, 95% CI [0.068; 0.203]), driven primarily by the reduced psychological friction of not physically handing over money [SBH24].

However, these systems come with a critical weakness: most of them depend on a reliable internet connection. And if there is one thing that festivals are known for, it is terrible connectivity. When tens of thousands of people gather in the same area, all using their phones simultaneously, the cellular network simply cannot handle the load. Shafiq et al. [SJL+13] conducted the first large-scale characterization of cellular network performance during crowded events using real-world traces from a

tier-1 US network. Their findings are striking: pre-connection failures increased by 100 to 5,000 times compared to routine days, driven by users exhausting the limited bandwidth of the signaling channel when too many devices attempt to acquire radio resources simultaneously. This degradation extended as far as 10 miles around the event venue as attendees arrived and departed. Even for users who managed to connect, voice call failures increased by 7 to 30 times, packet, and round-trip times increased by up to 7 times. The performance spikes were particularly severe during intermissions, when large numbers of attendees simultaneously attempted to use their devices. Notably, the study found that even with portable base stations and free Wi-Fi offloading already deployed, these degradation levels persisted, highlighting a fundamental capacity limitation rather than a simple configuration problem.

This is where things get problematic for payment systems. If the payment terminal at a food stall needs to contact a remote server to authorize a transaction, and the network is down or painfully slow, the transaction fails. The attendee cannot buy food. The vendor loses a sale. Multiply this by tens of thousands of transactions per day, and you can see why this is a serious issue. The most famous example of this going wrong happened at the Download Festival in the UK in 2015, the first major UK festival to go completely cashless. The RFID system experienced technical failures that left attendees unable to purchase food or drinks, with many taking to social media to describe the system as useless. Although organisers claimed only around one percent of attendees were affected, the backlash was severe enough that the festival abandoned the cashless-only approach the following year [eFe16].

The core question this thesis tries to answer is: can we build a payment system that works reliably at festivals, and that does not fall apart when the network has a bad moment? Our approach is to design a system that operates online by default, backed by a dedicated private Wi-Fi network deployed at the event venue. This private network serves as the primary connectivity layer, keeping vendor terminals connected to the cloud backend for real-time transaction processing. But because no network is perfectly reliable, especially one assembled in a field for a few days, the system also includes a modular offline extension that kicks in when connectivity is temporarily lost. The offline module is not the core of the architecture, it is a safety net. Certain operations, most notably wallet top-ups, are inherently online because they involve moving real money through external providers and cannot be handled locally.

1.1 Problem Statement

Festival environments present a unique combination of challenges that are not commonly addressed together in existing payment system literature. The starting point is connectivity. As discussed above, cellular networks collapse under the load of mass gatherings, and many festivals take place in rural or temporary venues where wired infrastructure simply does not exist. Our approach addresses this by deploying a dedicated private Wi-Fi network on site, which serves as the primary connectivity layer between vendor terminals and the cloud backend. Under normal conditions, the system operates fully online: transactions are authorized in real time against a central database, wallet balances are always up to date, and operations like top-ups that depend on external payment providers (Stripe, in our case) work without any issues. The private network effectively sidesteps the public cellular congestion that causes so many problems for existing solutions.

However, even a well-designed private network is not immune to failure. Equipment can malfunction, interference from the sheer density of wireless devices in a small area can cause temporary dead zones, and multi-day events put sustained stress on infrastructure that was assembled in a field a week earlier. When the network goes down, even for a few minutes during a peak period, the system still needs to process payments. This is where the offline extension module comes in. Rather than treating offline operation as the primary design goal, we treat it as a resilience layer: a modular component that activates when connectivity is temporarily lost and ensures that the festival economy does not grind to a halt. The key word here is temporarily, because prolonged offline operation introduces complications that are best avoided, as we will discuss below.

The second challenge is that these are closed ecosystems. Unlike a regular retail environment where a customer walks into a shop and pays with a bank card, a festival creates its own little economy. Attendees load credits, spend them at authorized vendors, and ideally get a refund for whatever they did not spend. This is what the payment industry calls a closed-loop system: the money circulates within a defined perimeter, controlled by a single operator, rather than flowing through the global banking network. The closed-loop nature is actually an advantage in our case, because it means we control the entire transaction flow and do not depend on external authorization networks that would be unreachable during a connectivity outage.

The third challenge is scale and speed. A major festival might have hundreds of vendor points, processing thousands of transactions per hour, especially during peak times like meal breaks or between performances. The payment system must be fast enough that it does not create bottlenecks. Attendees are not going to wait two minutes for a payment to go through when they just want a sandwich. In the

online mode, the cloud backend handles this load centrally. In the offline mode, each terminal processes transactions locally, which is actually faster per transaction but introduces the next challenge.

The fourth challenge is consistency during offline periods. When terminals lose connectivity and start processing transactions independently, they make local decisions about whether a payment is valid without being able to check with the central server or with each other. What happens when two terminals approve transactions that together exceed a user's balance? This is the classic double-spend problem, and it is one of the hardest issues to solve in distributed systems. For a festival payment system, refusing to process transactions because we cannot guarantee consistency is not an acceptable answer, because that means people cannot buy food. So during these offline windows, we choose availability and accept that some inconsistencies may occur that need to be resolved once connectivity is restored.

This leads to the fifth challenge: reconciliation. When terminals reconnect after an offline period, all the transactions they processed independently need to be merged back into the central system state. The synchronization protocol must ensure that no transactions are lost and that conflicts, such as a balance that went negative due to concurrent offline spending, are resolved in a deterministic way. Because our system is online-first, these reconciliation windows are expected to be relatively short and infrequent compared to a system that operates offline by default. But they still need to be handled correctly, because even a brief network outage during a peak period can generate hundreds of transactions that need to be synchronized.

It is also worth noting that not all operations can work offline. Wallet top-ups, for instance, require real-time communication with Stripe (or any other On Ramp provider) and the cloud backend, because actual money is moving from the attendee's bank account into the system. You cannot process that locally. This is an intentional design boundary: the offline module handles payments (debiting the wallet), but funding the wallet (crediting it) remains an online-only operation.

1.2 Objectives

The main objective of this thesis is to design and implement a minimum viable product (MVP) for a cloud-based, closed-loop payment system tailored to festival and event environments, with a modular offline extension that provides resilience during temporary connectivity loss. The concept of a minimum viable product, as defined by Ries [Rie11], refers to a version of a product that allows the team to collect the maximum amount of validated learning with the least effort. In our case, this means building enough of the system to demonstrate that the core architectural ideas work, without necessarily implementing every feature a production deploy-

ment would need. Throughout this work, we explore different design architectures and patterns, discussing their benefits and trade-offs in the context of festival environments, before settling on the approach that best fits the constraints of the problem.

Specifically, the objectives of this work are the following:

O1: Design a system architecture that operates online by default through a dedicated private Wi-Fi network deployed at the festival site, with all core payment operations processed in real time against a cloud backend. The architecture should also include a modular offline extension that activates automatically when connectivity is temporarily lost, allowing vendor terminals to continue processing payments locally.

O2: Implement a wallet-based payment service capable of processing debit and credit operations between consumer and vendor accounts. Wallet top-ups are handled via Stripe and require an active connection to the cloud backend. Debit transactions (payments at vendor terminals) should work both online, where they are authorized centrally, and offline, where they are processed locally until connectivity is restored.

O3: Explore and evaluate different strategies for keeping the on-site and cloud components in sync, particularly after periods of offline operation. The chosen approach should ensure that transactions processed during connectivity loss are eventually reflected in the central system and that conflicts are handled in a consistent way.

O4: Implement a core service that manages vendors, products, stock, users, and events, providing the administrative backbone of the system.

O5: Develop client applications for the three main user roles: a Consumer App for attendees to top up their wallet and make payments via QR or NFC, a Vendor App for merchants to manage products and accept payments, and an Admin Dashboard for event organizers to oversee the entire operation.

O6: Evaluate the proposed architecture against the specific requirements of festival environments, discussing the trade-offs between online and offline modes, the limitations of the current implementation, and the design decisions that shaped the system.

1.3 Related Work

The idea of building payment systems that work without constant connectivity is not new, but most of the existing work comes from domains that are quite different from festivals. Transit card systems like Suica, Oyster, and Octopus have been doing offline transactions for decades, storing balances directly on the card chip so

that gates can open without waiting for a server response. These systems are well studied and provide useful lessons about transaction speed and security pitfalls, but they operate in a controlled infrastructure with fixed terminals and predictable usage patterns, which is not exactly what a festival looks like.

On the software architecture side, the local-first movement, particularly the work by Kleppmann et al. [KWvHM19], lays out principles for applications that treat the local device as the primary copy and synchronize with the cloud when they can. Concepts like Conflict-Free Replicated Data Types offer formal guarantees about convergence in distributed systems, and event sourcing provides a natural model for logging transactions that happened offline and replaying them later. These ideas inform our design, though they do not map one-to-one onto payment systems, since money has the annoying property of being a conserved quantity.

On the commercial side, providers like Tappit, Intellitix, Weezevent, and Glownet have deployed cashless systems at hundreds of festivals, but their architectures are proprietary and essentially undocumented in academic literature. The failures we do know about, such as the Download Festival 2015 crash or the Danish Nets outage in 2024, highlight the consequences of getting this wrong, but they do not tell us much about how to get it right.

In short, the building blocks exist across transit systems, distributed systems theory, and commercial festival deployments, but nobody has put them together into a documented, open architecture designed for the specific constraints of festival environments. That is the gap this thesis tries to fill. A more detailed treatment of each of these areas follows in Chapter 2.

Chapter 2

Background and Related Work

This chapter provides the theoretical and practical background needed to understand the design decisions we make later in the thesis. We start by looking at closed-loop payment systems, which is the category our system falls into, and then move on to the architectural patterns that deal with offline operation and distributed data. We also review the existing commercial solutions for festival payments and discuss the academic literature on conflict resolution in distributed systems. The goal is to give the reader enough context to understand why we made the choices we did, and what alternatives we considered along the way.

2.1 Closed-Loop Payment Systems

Before going into the technical details, it is worth clarifying what we mean by a closed-loop payment system, since this concept is at the core of our architecture. In the payment industry, there are basically two types of systems: open-loop and closed-loop. An open-loop system is what most people use every day without thinking about it. When you pay with a Visa or Mastercard, the transaction goes through a network of banks, card schemes, and processors that connect merchants to customers across the entire world. The money flows through multiple intermediaries, each taking a small fee, and the whole thing relies on a global infrastructure that has been built over decades.

A closed-loop system is fundamentally different. As described by SDK.finance [SDK24], "closed-loop payment systems are financial arrangements that confine transactions to a specific network or ecosystem." In a closed-loop, the issuer and the acquirer are the same entity, which means a single operator controls the entire flow of money within a defined perimeter. Think of it like this: you load credits into the system, you spend those credits at authorized points, and whatever you did not spend, you can get back as a refund. The money never leaves the operator's ecosystem dur-

ing the transaction itself. The operator has full control over the rules, the fees, the authorization logic, and most importantly for our use case, the infrastructure.

There are several reasons why closed-loop systems are so attractive for festival environments, and the network situation is just one of them. The first and most obvious reason is the connectivity problem we discussed in Chapter 1. A festival is already a place where the network is going to have a bad time, so depending on external payment networks like Visa or Mastercard that need to reach authorization servers on the other side of the world is just asking for trouble. With a closed-loop system, all the authorization logic lives inside the operator's own infrastructure, so when the network has issues the system only needs to talk to itself.

The second reason is fees. Open-loop payment processing is not cheap. Every time someone pays with a credit or debit card, the merchant pays an interchange fee to the card issuer, a scheme fee to the card network, and a processing fee to the payment processor. For a small transaction like buying a beer at a festival, these fees can eat up a significant percentage of the sale. In a closed-loop system, the operator cuts out all these intermediaries. There is no interchange fee, no scheme fee, and no processor taking a cut of every transaction. The only point where external fees apply is when money enters the system through a top-up, which involves Stripe or another payment provider, but even then, almoast always these topup fees are lower due to negotiatiosn with the payment provider (that is most likely a strategic partner). Once the money is inside the closed loop, every subsequent transaction is free for the operator. For a festival processing tens of thousands of small transactions per day, this adds up to serious savings.

The third reason is perhaps the most interesting one, and it has to do with regulation. In the European Union, handling electronic money is a regulated activity that requires an e-money license under the Electronic Money Directive (EMD2) [Eur09]. Getting an e-money license is an expensive and time-consuming process that involves capital requirements, compliance obligations, and ongoing regulatory oversight. However, many festival payment systems operate in something of a regulatory gray area by using tokens or credits instead of displaying actual currency values. The idea is that when an attendee tops up their wristband or app, they are not depositing money into an electronic wallet in the regulatory sense. Instead, they are purchasing prepaid tokens that can be redeemed at participating vendors within the festival. This distinction matters because prepaid instruments issued for use within a limited network of service providers can fall under an exemption in the Payment Services Directive (PSD2) [Eur15], specifically the limited network exclusion. Under this exemption, if the payment instrument can only be used to acquire goods or services within a limited network of providers (like the vendors inside a festival), the operator may not need a full e-money or payment institution license. This is

also why many festival systems show balances in "tokens" or "credits" rather than in euros or dollars. Displaying the balance as "50 tokens" instead of "50 EUR" reinforces the narrative that these are not electronic money but rather prepaid vouchers for a specific set of services. Of course, this is a simplification and the legal situation varies by jurisdiction. Some regulators have taken a stricter view, and operators who process large volumes may still need to register or obtain a limited license. But the point is that the closed-loop, token-based model gives festival operators a much simpler regulatory path compared to running a full open-loop payment service.

The fourth reason is control. In a closed-loop system, the festival operator decides everything: transaction limits, refund policies, spending restrictions, which vendors are authorized, and how the settlement process works. There is no card scheme telling you what you can and cannot do. This level of control is particularly valuable for events where the operator needs to manage the entire economy, from how much people can top up to how refunds are handled after the festival ends. And the most important reason, of course, taking a percentage cut out of all the transactions inside the event without having to trust that vendors are honest, given that the only way to make a payment is through the operator's own system

Of course, there is one moment where the closed-loop system has to interact with the outside world, and that is when real money enters the system. When an attendee tops up their wallet, actual euros or dollars need to move from their bank account into the festival operator's account. This part requires an external payment provider like Stripe and an active internet connection. But once the money is inside the closed loop, all subsequent transactions are internal and can, in principle, be processed without any external dependency.

To understand where our system fits in the bigger picture, it helps to look at three existing categories of closed-loop payment systems that have dealt with similar challenges: transit cards, campus and event tokens, and the EMV offline authorization model.

2.1.1 Transit Card Systems

Transit card systems are probably the most successful example of closed-loop payments that work without constant connectivity. If you have ever used public transport in a major city, chances are you have used one. Examples include Suica in Tokyo, Oyster in London, or Octopus in Hong Kong. These systems have been around for decades, processing millions of transactions per day, and they offer some valuable lessons for anyone trying to build a payment system that needs to work fast and reliably.

The basic idea behind all of them is similar, at least in their original implementa-

tions. Shirakawa [Shi02] describes how Suica's FeliCa chip handles fare deduction locally at the gate through encrypted read-write operations, without needing a network round-trip. Chau and Poon [CP03] similarly describe Octopus as a "rechargeable contactless stored value smart card," meaning the balance lives on the chip itself. In both cases, the terminal communicates directly with the card, checks the balance, deducts the amount, and writes the updated balance back. FeliCa chips can complete this entire process in under 0.1 seconds. That said, these systems are not purely offline. Transaction logs are collected from terminals and uploaded to central servers periodically for reconciliation, fraud detection, and reporting. And newer versions, like the Suica 2.0 platform, have started moving fare calculation to a central server while still using the card chip for local read-write operations at the gate.

Suica is a good example to look at in detail. Launched by JR East in 2001, it was one of the first large-scale contactless payment systems in the world. Shirakawa [Shi02] describes how the system was designed to handle the rush hour crowds at Tokyo's train stations, where gates need to process passengers at a rate of about one person per second. The key to achieving this speed was keeping the authorization logic entirely local: the gate reads the card, checks the balance, deducts the fare, and opens. No network round-trip required. The card uses Sony's FeliCa chip [Son24b], a contactless IC with an embedded antenna that communicates using Manchester coding at 212 kbit/s in the 13.56 MHz range, requiring a proximity of 10 centimeters or less. It does not need a battery since it draws power from the reader when the card gets close enough, and the reader stops supplying power once the data transfer is done. Each transaction completes in approximately 0.1 seconds. Security is built into the protocol through mutual authentication: every time the card and reader communicate, a new encryption key is dynamically generated, which prevents fraud like impersonation or replay attacks. In 2011, Sony announced a new generation of FeliCa chips that adopted AES (Advanced Encryption Standard) encryption, and the newest generation announced in 2020 received the ISO/IEC 15408 EAL6+ certification, one of the highest security levels under the Common Criteria standard. Figure 2.1 illustrates how this mutual authentication works between the card and the gate reader.

The way mutual authentication works is basically a challenge-response protocol. The reader generates a random number and sends it to the card as a challenge. The card takes that number, encrypts it with the secret key that is stored inside the chip, and sends the encrypted result back. The reader already knows the key too, so it can check if the response is correct. If it is, the reader knows it is talking to a legitimate card and not a fake one. Then the process goes the other way around: the card sends its own random challenge to the reader, the reader encrypts it with the

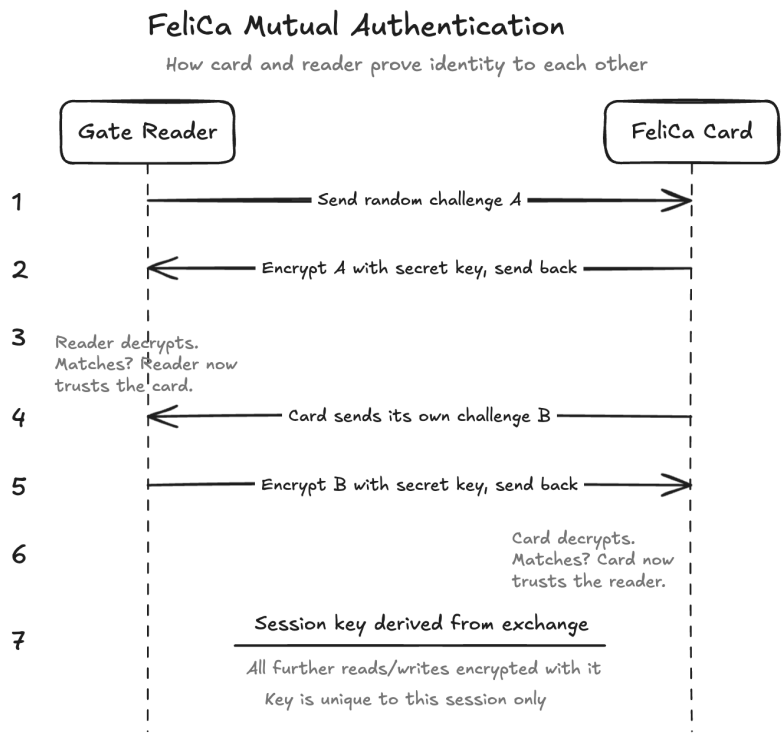

Figure 2.1: FeliCa mutual authentication flow between the gate reader and the contactless card. A new encryption key is derived for each session, making captured data useless for future transactions.

same key, and the card checks the response. Only after both sides have verified each other does the actual balance read and write happen. At the end of this exchange, both sides derive a session key that is used to encrypt everything that follows. This session key is unique to this one transaction and will never be reused.

This design also makes card cloning very hard to pull off. The secret key lives inside the chip's secure memory and it never leaves it. You cannot read it out, not even if you physically open the card. So even if someone has a reader and captures all the radio signals during a transaction, the encrypted responses they capture are useless for next time because the challenges will be different. To fake a valid response, an attacker would need the actual secret key, and there is no way to get it from the captured data alone. What about physical attacks, like putting the chip under a microscope and probing the circuits, or doing power analysis where you measure the chip's electricity consumption to try to figure out the key? That is exactly what the EAL6+ certification covers. To get that certification level, the chip has to be tested against these kinds of hardware attacks and prove it can resist them.

Octopus in Hong Kong follows a similar model but expanded the use case beyond transit. Launched in September 1997, it was actually the first contactless smart card system in the world to adopt Sony's FeliCa technology [Son24a], four years

before Suica. What started as a pure transit card evolved into a general-purpose stored-value card accepted at convenience stores, restaurants, parking garages, and even vending machines. By the early 2000s, it had over 11 million cards in circulation in a city of 7 million people, which gives you an idea of how widely adopted it became. The interesting thing about Octopus from our perspective is that it demonstrated that a closed-loop, card-stored-balance system could scale beyond its original domain while maintaining fast transaction times.

Now, what can we learn from these systems for our festival payment architecture? The first lesson is that speed matters a lot. Transit systems proved that sub-second transactions are not just nice to have, they are a hard requirement when you are processing large volumes of people who do not want to wait. At a festival, the same principle applies: nobody wants to stand in line for two minutes waiting for a payment to go through when they just want a beer.

The second lesson is about where the balance lives. Transit cards store the balance on the card chip, which is what makes offline operation possible. In a festival context, NFC bracelets can work in a similar way: the balance is written directly to the chip on the bracelet, and the vendor terminal reads and updates it locally, just like a Suica gate does with a FeliCa card. This means transactions can happen even without network connectivity. However, in our system we also support a smartphone app as an alternative to bracelets. With a phone, we cannot write balances directly to the NFC chip the way you can with a bracelet or a transit card. So for the app-based flow, the authoritative balance lives on the server, and during offline periods, vendor terminals work with cached snapshots of recent balances. This dual approach, bracelets with on-chip balances and phones with server-side balances, introduces its own set of challenges, which we will discuss in detail in Chapter 3.

The third lesson is about the limitations of the transit model. Transit systems operate in a very controlled environment: the terminals are fixed, maintained by professionals, connected to a private network, and the usage patterns are more or less predictable. A festival is a completely different story. Vendor terminals are mobile devices carried by staff who may not be very technical, the network is a temporary Wi-Fi setup in a field, and usage spikes happen unpredictably during meal breaks or between performances. So while transit systems show that offline closed-loop payments can work, their specific solutions do not transfer directly to our context.

2.1.2 Festival Payment Systems

Over the past decade, a whole industry has grown up around providing cashless payment solutions for festivals and events. Companies like Tappit, Intellitix, Weezevent,

and Glownet have deployed their systems at hundreds of events worldwide. The typical setup works like this: attendees receive an RFID wristband when they enter the festival. They load credits onto the wristband using cash top-up stations or an online portal before the event. At each vendor point, there is a handheld terminal that reads the wristband, deducts the purchase amount, and writes the new balance back to the chip. It is essentially the same model as transit cards, adapted for the festival context.

These systems have proven that cashless festivals are commercially viable. As mentioned in Chapter 1, Tappit reports that RFID systems reduce transaction times by up to 80 percent compared to cash and increase average spending by about 22 percent [Tap24]. The speed improvement comes from eliminating the need to count cash and make change. The spending increase is attributed to the psychological effect of not physically handing over money, which is consistent with the broader academic literature on cashless payment behavior [SBH24].

However, there are some significant limitations to the RFID wristband approach that motivated us to explore a different path. First, the RFID chips used in most festival wristbands have very limited storage and processing capability. They can store a balance and maybe a transaction counter, but they cannot run complex logic. This means that all the intelligence has to be in the terminal, and the terminal needs to handle things like transaction validation, balance updates, and security checks. If the terminal loses connectivity to the backend and its local cache is stale or incomplete, it has a problem.

Second, and this is the bigger issue, many of these RFID systems rely on MIFARE Classic chips, which have well-documented security vulnerabilities. Garcia et al. [GdKGM+08] demonstrated practical attacks against MIFARE Classic's proprietary CRYPTO1 cipher, showing that the encryption could be broken and the card contents cloned or modified. For a payment system, this is a serious problem because it means that a sufficiently motivated attacker could tamper with their balance. More modern chips like MIFARE DESFire have addressed these vulnerabilities, but they are more expensive, and many festival deployments still use the older technology to keep costs down.

Third, the architectures of these commercial systems are proprietary and essentially undocumented. There is no published academic analysis of how Tappit or Intellitix handle offline operation, conflict resolution, or balance reconciliation. This makes it hard to learn from their design decisions or to identify best practices. All we have are the occasional failure reports, like the Download Festival incident mentioned in Chapter 1, which tell us what went wrong but not how things were supposed to work.

Since there is not a lot of published research on how these systems work in prac-

tice, we talked to people who were involved in deploying payment infrastructure at Untold, one of the largest music festivals in Romania. From those conversations we learned that the typical approach is to deploy a local network on-site and bring dedicated servers to the venue, so that the core payment logic runs locally and does not depend on a remote cloud. Even with this setup though, the network is still unreliable enough that top-up operations can take two to four hours to synchronize across all terminals when the Wi-Fi coverage gets bad. This is a real problem for attendees who just loaded money and want to spend it immediately but the vendor terminal does not know about it yet. It shows that maintaining reliable Wi-Fi coverage is not just something nice to have, it is a core infrastructure requirement for any festival payment system. We discuss how our architecture handles these synchronization delays in Chapter 3.

Given these trade-offs, there are two main directions we could take. One option is to stick with the RFID wristband model and use NFC bracelets with on-chip balances, which gives us the advantage of fully offline transactions like transit cards but limits what we can do on the user side. The other option is to go with a smartphone app as the consumer device, where the balance lives on the server and payments happen via QR codes or NFC between the phone and the vendor terminal. A phone-based approach gives more flexibility for things like showing transaction history, doing top-ups from the app, and pushing notifications, but it means we depend more on the network since the balance is not stored locally on a chip. There is also a middle ground where we support both. We explore these options further in Chapter 3.

2.1.3 EMV and Offline Authorization

The EMV standard, named after its original creators Europay, Mastercard, and Visa, is the technology behind the chip cards that most people carry in their wallets. While EMV is an open-loop standard designed for the global payment network, it includes a mechanism for offline transaction authorization that is worth understanding because it addresses a similar problem to ours: how do you authorize a payment when you cannot reach the authorization server?

The way EMV offline authorization works is through a set of risk parameters stored on the card chip. The card knows its own balance limits, the maximum amount for a single offline transaction, the maximum number of offline transactions allowed before the card must go online, and the total cumulative amount that can be spent offline. When a merchant terminal cannot reach the bank's authorization server, it can still approve transactions by checking these parameters locally. If the transaction falls within the allowed limits, it gets approved offline. If it ex-

ceeds them, the terminal declines the transaction or forces an online authorization attempt.

This is a clever approach because it distributes the risk. The card issuer decides how much offline risk they are willing to accept for each cardholder, and those limits are baked into the card at the time of issuance. A premium cardholder might have higher offline limits than a basic cardholder. The merchant also has some say in this: their terminal can have its own floor limit below which it will always accept transactions without going online.

However, there is a well-known problem with EMV offline authorization: the system trusts the card to honestly report its own state. Murdoch et al. [MDAB10] demonstrated a man-in-the-middle attack on the EMV protocol where a device placed between the card and the terminal could trick the terminal into believing that a PIN had been correctly verified when it had not. While this specific attack targets the PIN verification rather than the offline balance logic, it illustrates a broader point: any system where the authorization decision depends on data stored on a device in the user's possession is vulnerable to tampering if the security of that device is compromised.

For our system, the EMV offline model offers both inspiration and a warning. On the inspiration side, the idea of using spending limits to control offline risk is directly applicable. We implement something similar: during offline periods, vendor terminals enforce a maximum transaction amount and a maximum total offline spend per consumer, which limits the potential damage from double-spending or stale balance snapshots. On the warning side, the EMV experience shows that offline authorization always involves a trade-off between availability and security. The more you allow offline, the more risk you accept. Our system handles this by keeping offline periods as short as possible and treating offline mode as a temporary fallback rather than a primary operating mode.

2.2 Offline-First Architecture Patterns

When you build a system that needs to work without a network connection, you are basically deciding where on a spectrum your application sits. On one end you have fully online systems, where every operation needs a round-trip to a server and nothing works without internet. On the other end you have fully offline systems, like the FeliCa transit cards we discussed earlier, where everything happens locally and the network is only used for background synchronization. Most real-world applications fall somewhere in between.

For our festival payment system, we need to be somewhere in the middle of this spectrum. The system should work online by default, because that gives us the

strongest guarantees about balance correctness and transaction ordering. But when the network goes down, and at a festival it will go down, the system needs to keep processing payments using whatever data it has cached locally. This means we need architectural patterns that support both modes of operation and can switch between them without losing data or creating inconsistencies that cannot be resolved later.

In this section we look at four architectural patterns that are relevant to this problem. Each one addresses a different aspect of building systems that can work without constant connectivity.

2.2.1 Local-First Software

The term "local-first software" was introduced by Kleppmann et al. [KWvHM19] in a 2019 paper that argued for a different way of thinking about where data lives in an application. The basic idea is that the primary copy of the data should be on the user's local device, not on a remote server. The cloud still exists, but its role is to sync data between devices rather than to be the single source of truth. If the server goes down, or the network disappears, the application keeps working because the data is right there on the device.

Kleppmann and his co-authors defined seven ideals for local-first software: no spinners (the app should be fast because it reads from local storage), the user's work should be available on multiple devices, the network should be optional, collaboration should be seamless, the data should be accessible for a long time (not locked behind a service that might shut down), security and privacy should be built in, and the user should have full ownership and control over their data.

Now, our festival payment system does not fit neatly into the local-first model, and there is a good reason for that. In a collaborative document editor, if two people edit the same paragraph and the results get merged a bit awkwardly, that is annoying but not catastrophic. In a payment system, if two terminals both think a user has 50 euros and each one authorizes a 40 euro purchase, you have just lost 30 euros. Money is a conserved quantity. You cannot merge two versions of a balance the way you merge two versions of a text document.

That said, some of the local-first principles are directly useful for us. The idea of keeping a local copy of the data so the app does not freeze when the network drops is exactly what our vendor terminals need to do. The difference is that in our case, the local copy is a read cache with limited write authority, not the primary copy. The server remains the source of truth, and offline transactions are treated as provisional until they get confirmed by the server. This is a weaker version of local-first, but it is the right trade-off for a system where financial correctness matters more than collaboration flexibility.

2.2.2 Conflict-Free Replicated Data Types (CRDTs)

CRDTs, or Conflict-Free Replicated Data Types, are data structures specifically designed for systems where multiple nodes can modify the same data independently, without coordinating with each other, and still end up in a consistent state. They were formalized by Shapiro et al. [SPBZ11] in 2011, and the key property that makes them work is that all operations on the data structure are designed to be commutative, associative, and idempotent. What this means in practice is that it does not matter what order the updates arrive in, or if some updates arrive more than once. The result will always be the same.

The simplest example is a G-Counter (grow-only counter). Imagine you have five terminals at a festival, and each one counts how many transactions it has processed. Each terminal only increments its own counter. When they sync, you just take the maximum value for each terminal's counter and sum them up. No conflicts possible. A PN-Counter extends this idea by having two G-Counters, one for increments and one for decrements, and the actual value is the difference between the two.

This sounds like it could work for tracking balances, but there is a catch. A balance is not just a counter that goes up and down freely. Money is conserved: you cannot spend more than you have. CRDTs are great at ensuring convergence (everyone ends up with the same value), but they do not enforce invariants like "the balance must not go below zero." If two terminals independently authorize purchases against the same balance while offline, a naive CRDT approach would let both go through, even if the total exceeds the available balance. You would only discover the overspend after synchronization.

Kleppmann [KB17] explored this limitation in the context of JSON CRDTs and noted that while CRDTs guarantee convergence, they do not guarantee that the converged state is one that any individual node would have accepted. For payment systems, this is a real problem. We cannot just let every offline transaction go through and hope the math works out later.

So for a payment system, pure CRDTs are not enough on their own. You need something extra on top to deal with the fact that balances have constraints. There are a few ways to handle this. One option is to record offline transactions as events and have the server replay and validate them when connectivity returns, rejecting any that would overdraw the balance. Another option is to pre-allocate spending limits per terminal so that each terminal can only authorize up to a fixed amount while offline, which reduces the chance of overspending in the first place. A third option is some combination of the two. We explore these trade-offs further in Chapter 3.

2.2.3 Event Sourcing and CQRS

Event sourcing is a pattern where instead of storing the current state of an object (like "balance = 47.50"), you store the entire sequence of events that led to that state ("topped up 50.00, bought beer for 2.50"). The current state can always be reconstructed by replaying the events from the beginning. Fowler [Fow05] describes event sourcing as a way to "ensure that all changes to application state are stored as a sequence of events," and one of the main benefits is that you get a complete audit trail for free.

This pattern fits really well with offline payment processing. When a vendor terminal goes offline, it cannot update the central database directly. But it can keep recording events locally: "user X paid 5.00 for item Y at timestamp Z." Each event is self-contained and immutable. When the terminal reconnects, it uploads its event log to the server, and the server can process these events in order to update the authoritative state. If there is a conflict (say, the user ran out of money between two offline purchases), the server can detect it during replay and handle it according to whatever policy the operator has defined.

CQRS, which stands for Command Query Responsibility Segregation, is a related pattern that is often used together with event sourcing. The idea, as described by Young [You10], is to separate the write side of the system (commands that modify state) from the read side (queries that return data). In a traditional system, the same data model handles both reads and writes. In CQRS, you have one model optimized for processing writes (accepting payment events) and another model optimized for fast reads (showing the current balance on a screen).

For our system, this separation is useful because the write side and the read side have very different requirements. On the write side, we need strong ordering guarantees and conflict detection. On the read side, vendor terminals need to quickly look up a user's balance to decide whether to authorize a payment, and this lookup needs to work even when the terminal is offline. By separating these concerns, we can have the read model work from a local cache while the write model queues events for later synchronization, without the two getting in each other's way.

2.2.4 Edge Computing

Edge computing is the idea of moving computation closer to where the data is generated, instead of sending everything to a central server or cloud for processing. Shi et al. [SCZ+16] describe it as "pushing the frontier of computing applications, data, and services away from centralized nodes to the logical extremes of a network." In practice, this means doing as much work as possible on the device that is closest to the user, and only involving the cloud when necessary.

In the context of our festival payment system, the vendor terminals are the edge nodes. Each terminal runs a local instance of the payment logic: it can read a user's balance from its local cache, validate a transaction against the cached data, record the transaction as an event, and update its local view of the balance. All of this happens on the terminal itself without any network communication. The cloud server (or the on-site local server, as we discussed in the context of the Untold festival) acts as the central coordinator that collects events from all the edge nodes, resolves conflicts, and pushes updated state back out.

This works well for festival environments because the network is the weakest link in the whole system. The less you depend on it for individual transactions, the better. If the terminal can handle a payment entirely on its own, then a network outage does not mean a payment outage. It just means that the synchronization of data gets delayed. The trade-off, of course, is that edge nodes are working with potentially stale data. A terminal that has been offline for 30 minutes might not know about a top-up that happened 20 minutes ago. How we handle this staleness, and what limits we put on edge-processed transactions to manage the risk, is something we cover in detail in Chapter 3.

2.3 Conflict Resolution in Distributed Systems

When multiple vendor terminals at a festival are processing payments independently, without being able to talk to each other or to a central server, they are going to end up with different views of the world. Terminal A might think a user has 50 euros because it processed a top-up, while Terminal B still shows the old balance of 20 euros because it has not received the update yet. If both terminals authorize purchases during this window, the system will have conflicting records that need to be sorted out once everyone reconnects.

This is not a new problem. Distributed systems have been dealing with conflicting state for decades, and there is a solid body of theory and practice around how to handle it. In this section we go through the main concepts that are relevant to our case: what trade-offs are fundamental and cannot be avoided, how systems achieve consistency without requiring all nodes to agree in real-time, how to figure out the order of events when there is no shared clock, and how to let nodes work independently with the plan of fixing things up later.

2.3.1 The CAP Theorem

In 2000, Eric Brewer presented a conjecture at the ACM Symposium on Principles of Distributed Computing that would become one of the most cited results in dis-

tributed systems. The conjecture, which Gilbert and Lynch [GL02] formally proved in 2002, says that a distributed system can only guarantee two out of three properties at the same time: Consistency (every node sees the same data at the same time), Availability (every request to a non-failing node gets a response), and Partition Tolerance (the system keeps working even when the network between nodes is broken).

The important word here is "partition tolerance." In a festival environment, network partitions are not an edge case, they are a regular occurrence. When the Wi-Fi drops between two parts of the venue, the vendor terminals on each side are partitioned from each other and from the server. At that point, according to CAP, we have to choose: do we want consistency or availability?

If we choose consistency, then when a terminal cannot reach the server to verify a balance, it should refuse the transaction. The data stays consistent, but the system is unavailable and the attendee cannot buy anything. If we choose availability, the terminal goes ahead and authorizes the payment based on its local cache, even though it cannot be sure the data is up to date. The payment goes through, the attendee is happy, but we might end up with inconsistencies that have to be fixed later.

For a festival payment system, refusing transactions because the network is down is not really an option. Nobody is going to accept a system that stops working every time the Wi-Fi gets flaky, which at a festival is all the time. So during network partitions, our system leans toward availability. Transactions keep going through, and consistency is restored after the partition heals. This is not a unique choice. Many real-world systems choose availability during partitions and then run a reconciliation process afterward. That is essentially what we are doing.

2.3.2 Eventual Consistency

If we are giving up strong consistency during network partitions, the question becomes: what kind of consistency can we actually promise? The answer, for most distributed systems that prioritize availability, is eventual consistency. Vogels [Vog09] defines it simply: if no new updates are made to a piece of data, then eventually all replicas will converge to the same value.

The word "eventually" is doing a lot of work in that definition. It does not say when the replicas will agree, just that they will at some point. In practice, this usually happens pretty fast once connectivity is restored. But during the partition, different nodes can have different views of the data, and that is considered acceptable.

The most famous real-world example of eventual consistency is Amazon's Dynamo, described by DeCandia et al. [DHJ+07]. Dynamo is a key-value store that Amazon built for their shopping cart and other services that need to be available

all the time, even during failures. Dynamo allows writes to happen on any node, even if that node cannot reach the others. When the partition heals, Dynamo uses application-specific logic to merge conflicting versions.

For our system, eventual consistency means the following: while terminals are offline, they work with their local data and record all transactions locally. When they reconnect, they send their transaction logs to the server. The server processes everything, figures out the correct state of each balance, and pushes the updated data back to all terminals. At that point, everyone is consistent again. The window during which terminals might have stale or conflicting data is the offline period, and the goal of the architecture is to make this window as short as possible and to limit the damage that can happen during it (for example, by capping how much a terminal can authorize while offline).

2.3.3 Transaction Ordering and Why Timestamps Are Enough

One of the classic problems in distributed systems is figuring out the order in which things happened across different nodes. The standard solutions from the literature are logical clocks and vector clocks, which use counters instead of real time to establish ordering. These are necessary in systems like high-frequency trading platforms, where events happen microseconds apart on different machines and physical clock drift is larger than the interval between events.

However, for a festival payment system, the situation is fundamentally different. Transactions happen at human speed. A person walks to a vendor, waits in line, orders something, taps their wristband. The gap between one transaction and the next, even for the same user, is measured in minutes, not microseconds. Even cheap NTP synchronization keeps clocks within a few hundred milliseconds of each other, and that kind of drift is completely irrelevant when the events we care about are minutes apart.

This means that plain wall-clock timestamps, recorded by each vendor terminal at the moment of the transaction, give us unambiguous ordering in practice. There is no realistic scenario where two transactions from the same user happen so close together that clock drift changes the order. The physical constraint of the wristband model reinforces this: a user can only be at one vendor at a time, so transactions from the same user are naturally serialized by their physical presence.

The practical advantage of using timestamps over vector clocks is significant. With timestamps, vendor terminals do not need to communicate with each other at all during offline periods. Each terminal just records its local time with each transaction. When the network comes back, everyone pushes their logs to the server, the server sorts by timestamp, and we are done. There is no vector state to main-

tain, no counter synchronization between devices, and no extra complexity in the protocol. For a system that needs to work reliably in a festival environment where connectivity is unreliable, this simplicity is a real advantage.

Chapter 3

System Architecture

This chapter goes through the architecture of the closed-loop payment system. We start with the big picture, explain why we chose this approach over the alternatives, and then go into each component one by one. The whole idea behind the design is to keep things simple: one server, one app that works for everyone, and strive to have always network support. If network however will go down the balance will be always cached on the bracelet and used as a deciding factor if the payment will be approved or not. The system should be easy to deploy, easy to understand, and tough enough to handle what actually happens at a festival.

3.1 High-level overview

Most existing festival payment systems need a technical team to physically go to the venue, set up local servers, deploy a private network, and babysit the infrastructure for the entire duration of the event. As we discussed in Section 2.1.2, the standard approach at festivals like Untold involves bringing dedicated servers on-site so that the payment logic runs locally. This makes sense when the balance lives on a remote server and every transaction needs a network round-trip to authorize. In that model, if the connection between the venue and the cloud drops, nobody can pay for anything, so you bring the server to the venue to avoid that dependency.

Our system takes a different path. We aim to always have reliable internet access using Starlink as the primary uplink, paired with a redundant network topology that keeps terminals talking to the server even when individual links degrade. The wristband-as-cache fallback exists for the residual edge cases, but the goal is to never need it.

This means there is no need to deploy anything at the festival venue. No local servers, no private network infrastructure, no IT team on-site. The organizer creates an event through the admin dashboard, configures the parameters, approves

the vendors, and distributes the wristbands. The vendors download the app, get whitelisted, and they are ready to accept payments. The server runs in the cloud, the same way it runs for every other festival on the platform. Nothing changes in the deployment.

The system has four main components: a backend server, a mobile application for attendees and vendors, a web-based admin dashboard, and NFC wristbands that carry the actual token balance for each attendee. The server is a single, multi-tenant service that manages all festivals from one deployment. There is no need to spin up a separate server or database for each event. An admin creates an event, configures it, and the system handles the rest. The mobile app is what attendees and vendors interact with: attendees see their balance and history, and vendors use it as a soft point-of-sale terminal. Admins manage the event through a separate web dashboard that works in any browser. The wristband is where the balance actually lives during the event.

3.1.1 The hybrid reconciliation model

The core architectural decision is how the wristband and the server share responsibility for the balance. We chose what we call **Approach H, the hybrid model**: credits flow through the server and debits happen on the chip. Each side owns one direction, and the two flows never collide.

The state is split across the two sides. Each wristband stores three fields per event: `balance` (the cached value the chip can spend from), `debit_counter` (a monotonic counter the chip increments on every payment), and `credit_counter_seen` (the highest server-side credit sequence the chip has acknowledged). The server keeps the mirror image, again per wristband per event: `balance` (the canonical value), `debit_counter_seen` (the highest debit counter it has reconciled from terminals), and `credit_counter` (incremented on every server-side credit). The balance is event-scoped, not user-scoped, so the same person attending two festivals has two independent wallets and two independent counter pairs.

For this thesis demo we use a **plain NTAG-class NFC chip** without a secure element. The hybrid model does not depend on chip-side cryptography to work: it only needs the chip to store a few fields, increment a counter atomically, and keep what was last written to it. The security implications of using normal NFC, and the upgrade path to authenticated chips, are discussed in Chapter 5.

Figure 3.1 shows the high-level architecture. The thing to notice is the NFC connection between the Vendor POS and the wristband at the bottom: that is the actual debit path, and it does not go through the server. Everything above (the load balancer, the server instances, the database) handles management, reporting, top-ups,

and the periodic reconciliation that brings the two sides back in sync.

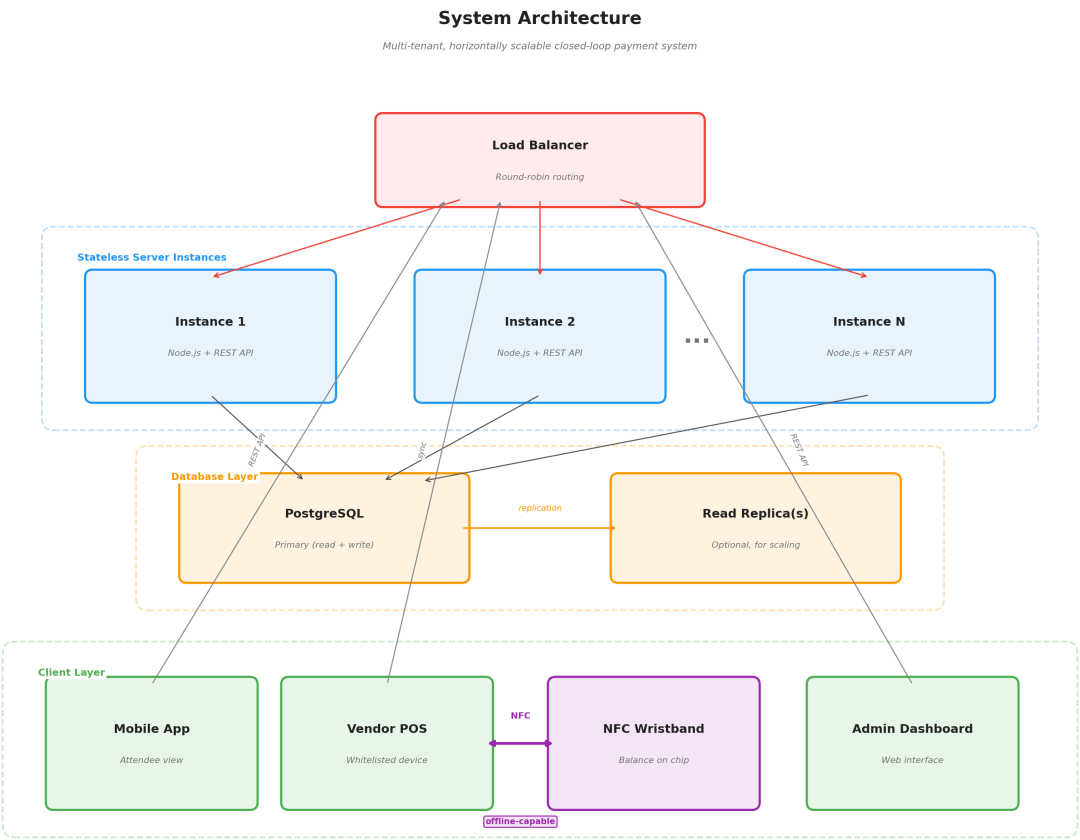

Figure 3.1: High-level system architecture. Debits happen directly between the vendor device and the NFC wristband. Credits go through the backend, which is also responsible for top-ups, sync, and reporting. Server instances are stateless and horizontally scalable behind a load balancer.

3.2 Balance reconciliation model

A closed-loop payment system with offline-capable terminals has two writers that can mutate the balance: the server (when users top up) and the chip (when users pay vendors). During a network partition both can act on their own, and once connectivity is back the system has to converge on a correct final value. Naive solutions tend to either double-spend, double-credit, or lose money in subtle ways. Our hybrid model handles this by keeping the two writers in separate lanes that never overlap.

3.2.1 State on each side

The wristband stores three reconciliation fields per event, plus identity fields written at provisioning:

- `balance`: the cached balance in minor units, what the chip can spend from when offline.
- `debit_counter`: a monotonic counter the chip increments by one on every chip-side debit. The chip is the only writer.
- `credit_counter_seen`: the highest server-side credit sequence number the chip has acknowledged. Updated whenever a terminal writes a new triple back to the chip.
- identity fields `bracelet_uid` and `event_id`, written once at provisioning.

The server stores the mirror image, per wristband per event:

- `balance`: the canonical balance.
- `credit_counter`: a monotonic counter the server increments by one on every server-side credit. The server is the only writer.
- `debit_counter_seen`: the highest `debit_counter` value the server has reconciled from any terminal.

The two counters are independent monotonic sequences, each with one writer. The chip owns `debit_counter`; the server keeps a read-only mirror in `debit_counter_seen`. The server owns `credit_counter`; the chip keeps a read-only mirror in `credit_counter_seen` Because each lane has exactly one writer, there is no merge conflict to resolve within a lane. The balance is event-scoped: the same person at two festivals has two independent wallets.

For this thesis demo the wristband is a plain NTAG-class NFC chip. The hybrid model does not depend on chip-side cryptography. The security implications of normal NFC, and a future hardening path to DESFire EV2/EV3 with AES-128 mutual authentication, are discussed in Chapter 5.

3.2.2 Invariants

The correctness of the model rests on three invariants that must hold at all times.

1. **The chip never increases its own balance.** Every chip-side mutation is a debit: `balance -= amount` together with `debit_counter += 1`, applied as one atomic write.
2. **The server is the only source of credits.** Top-ups, refunds, and operator adjustments mutate the server's balance and increment `credit_counter`. The chip picks up these credits passively on the next tap by an online terminal.

3. **Counters are monotonic with a single owner each.** `debit_counter` only ever grows on the chip; `credit_counter` only ever grows on the server. The mirror fields (`debit_counter_seen` and `credit_counter_seen`) only ever grow toward the owner's value.

As long as these three hold, the chip and the server cannot disagree about who added money or who spent it. Reconciliation reduces to taking the union of two non-conflicting streams.

3.2.3 The four operations

Top-up. Always online. The server checks the payment, increments `balance` and `credit_counter`, writes a transaction row, and acknowledges. The chip is not touched. The chip is now stale relative to the server, but that is harmless: the next tap reconciles it.

Online debit. The terminal reads the chip and forwards the request to the server with the chip's current `debit_counter` and `credit_counter_seen`. The server checks for pending credits (any sequence number greater than `credit_counter_seen`) and applies them as part of the same response. It then validates the debit, updates its ledger, and returns the new triple `(balance, debit_counter, credit_counter_seen)` to the terminal, which writes it atomically to the chip. Pending credits and the new debit are picked up in one round trip.

Offline debit. The terminal reads the chip, checks that the balance is enough, and performs an atomic decrement on the chip. It generates an idempotency key, queues the transaction locally with the resulting `debit_counter` value, and serves the customer. The terminal keeps draining its queue in the background as connectivity comes and goes.

Sync. When a terminal regains connectivity, it pushes queued transactions to the server in batches. The server applies them idempotently using the per-transaction key, updates `balance`, and advances `debit_counter_seen` to the highest counter value observed. Because debits are subtraction and the chip already enforced no-overdraft at the moment of each debit, the server can apply queued transactions in arrival order without strict sequencing.

3.2.4 Worked example

Consider the case where a top-up and an offline debit happen at the same time. The starting state is chip `(balance=50, debit_counter=0, credit_counter_seen=0)` and server `(balance=50, credit_counter=0, debit_counter_seen=0)`.

The user pays 10 tokens at an offline terminal. The chip becomes `(40, 1, 0)` and the terminal queues a debit of 10 with `debit_counter=1`. The server has not heard about this yet.

In parallel, a partner pushes a 20-token top-up through the server. The server becomes `(70, 1, 0)`. The chip has not heard about this yet either.

The user then taps an online terminal somewhere else. The terminal reads `(40, 1, 0)` from the chip and forwards it to the server. The server sees that `credit_counter=1` is greater than the chip's `credit_counter_seen=0`, so there is one pending credit of 20 tokens to apply. It also sees that `debit_counter=1` is greater than `debit_counter_seen=0` so there is one chip-side debit it has not received yet, but it does not need the amount to apply the credit safely. The server writes `(60, 1, 1)` to the chip.

When the offline terminal eventually syncs the queued debit of 10, the server applies it: balance goes from 70 to 60, and `debit_counter_seen` advances from 0 to 1. Both sides now sit at `balance=60` with `debit_counter=1` and `credit_counter=1`. No conflict resolution was needed because the two streams never collided.

3.2.5 Lost transactions

The model does not magically prevent loss. If a terminal dies before draining its queue, the vendor has served the customer and the chip has correctly subtracted the amount, but the server has no record. The gap shows up automatically: when the server observes that `chip.debit_counter` exceeds `debit_counter_seen` plus the count of pending queued transactions tracked server-side, the difference represents lost debits.

The resolution policy is to trust the chip's balance immediately, wait a configurable grace period for late syncs, and after that reconcile by writing the chip's balance to the server and logging the discrepancy for ops review. The festival absorbs the loss for those specific transactions; vendors with local receipts can claim manually if the amount is worth chasing. The exposure is bounded by the sync interval, on the order of low hundreds of euros per dead terminal at festival rates, which is an operational risk rather than a correctness bug.

3.2.6 Comparison with alternatives

Before settling on the hybrid model we looked at eight different ways to split the balance between the chip and the server. Table 3.1 summarises them.

We picked H for three reasons. First, the mental model is the cleanest: each side owns one direction, so there is nothing to merge in the common case. Second, the lane separation means addition and subtraction commute, so the system stays

Approach	Writer model	Offline	Conflicts
A. Pure server	Server only	None	None
B. Server-wins cache	Server + terminal cache	Partial	Cache divergence
C. Single counter	Chip writes balance + counter	Full	Hidden divergence on
D. Two counters (chip-only)	Chip writes both	Full	Mostly avoided
E. Event log on chip	Chip stores event list	Full	Replay merge
F. Chip-wins	Chip is canonical	Full	Server reconstructs
G. PN-counter CRDT	Both sides increment per-actor counters	Full	CRDT merge
H. Hybrid	Server credits, chip debits	Full	None in common case

Table 3.1: Eight approaches to balance reconciliation between the chip and the server, evaluated for offline support, conflict handling, complexity, and loss profile.

correct under any interleaving of offline debits and online credits without needing vector clocks or CRDTs [SPBZ11]. Third, this lane separation makes the lost-transaction case from the use cases (UC6) trivial to detect: a single integer comparison between `chip.debit_counter` and `debit_counter_seen` tells you exactly how many debits the server is missing.

The other approaches all have a sharp edge somewhere. A and B are not viable for a festival in a field. C has a known correctness bug where chip and server can hold equal counters but divergent balances after concurrent operations in different directions. D, F, and E all give one side too much authority and create an asymmetry that bites later. G works but pulls in a CRDT runtime to solve a problem that does not really exist once you split the writers.

3.3 Lessons from related systems and patterns

The hybrid model did not appear out of thin air. It mixes ideas from transit cards, commercial festival systems, EMV, and the distributed systems literature. This section walks through what we kept and what we left behind.

3.3.1 What we learned from existing systems

In Chapter 2 we looked at transit cards, commercial festival payment systems, and the EMV offline authorization model. Each one shaped a different part of the design.

Transit card systems like Suica and Octopus (Section 2.1.1) are the proof that an on-chip balance works at scale. Suica processes millions of transactions per day with sub-second speed because the balance lives on the FeliCa chip and the gate does not need to call any server. Our wristband borrows that core idea: the chip carries the spendable balance, the terminal reads it, debits, and writes back. The difference is that Suica runs on fixed gates maintained by professionals in train stations with

dedicated private networks, while we run on regular phones carried by festival vendors in a field. So the principle transfers, but the implementation details are quite different. Suica is also chip-only on the spending side and back-office on the credit side, which is the same split we use in the hybrid model.

From the commercial festival systems like Tappit and Intellitix (Section 2.1.2), we mainly learned what we do not want to do. These systems require an IT team to go to the venue, set up local servers, deploy a private network, and manage the whole thing for the duration of the event. That works for a company with dedicated operations staff, but it does not work if the goal is a system that a small organizer can deploy by themselves. The conversations we had with people who worked on the Untold deployment confirmed this: even with dedicated on-site infrastructure, top-up synchronization delays of two to four hours were common when the Wi-Fi got bad. Our architecture avoids this entirely because there are no on-site servers to maintain. The chip handles offline debits, and the cloud server only needs to be reachable for top-ups and sync.

The EMV offline authorization model (Section 2.1.3) gave us the idea of using spending limits as a risk control. EMV cards carry parameters that define how much they can spend offline before they have to go online for authorization. We use a similar concept: the event admin can configure maximum transaction amounts and a maximum total offline spend per wristband. This is mostly a safety net for the edge case where someone does manage to tamper with a chip, it limits the damage. EMV also has the well-known weakness Murdoch et al. [MDAB10] showed, where the card self-reports its state and the terminal has to trust it. We discuss the equivalent question for our plain-NFC chip and the path to authenticated chips in Chapter 5.

3.3.2 What we took from the architectural patterns

In Section 2.2 we covered several architectural patterns for offline operation: local-first software, CRDTs, event sourcing with CQRS, and edge computing. Not all of them ended up being useful for our case.

Local-first software (Section 2.2.1) is a nice philosophy, but as we discussed in Chapter 2, it does not fit payment systems well. Kleppmann et al. [KWvHM19] designed it for collaborative applications where merging conflicting edits is annoying but acceptable. With money, you cannot just merge two conflicting balance updates and hope for the best. Our system borrows the basic idea of keeping a local copy of data so the app works without the network, but the server remains the source of truth and offline transactions are treated as provisional. It is a weaker version of local-first, and that is the right call for a system where financial correctness actually matters.

CRDTs (Section 2.2.2) [SPBZ11] are an interesting solution for data convergence in distributed systems, but they turned out to be overkill for what we need. CRDTs guarantee that all replicas converge to the same value regardless of the order updates arrive, which sounds great until you realize they cannot enforce constraints like "the balance must not go below zero." The hybrid model makes the convergence problem trivial without a CRDT runtime: each side writes only to its own lane, and the merge is just addition and subtraction with one monotonic counter per lane. A PN-counter would work, but it would carry per-actor state and a generic merge function for a problem that lane separation already solves with two integers.

Event sourcing (Section 2.2.3) is the one pattern we actually use directly. Every transaction that happens on a vendor device is recorded as an immutable event with a timestamp, a device ID, a wristband ID, and the amount. When the device is online, these events are sent to the server immediately. When it is offline, they queue up locally and get pushed as a batch when connectivity returns. The server reconstructs the state of each wallet by replaying these events. This gives us a full audit trail of everything that happened, which is important for settlement and for the event organizer to see where the money went. We do not use CQRS in the formal sense (separate read and write models), but in practice the vendor app does maintain a local read cache of recent transactions that is separate from the write queue, which is a similar idea.

Edge computing (Section 2.2.4) shows up at two levels. The first and more obvious one is the vendor phones. Each phone runs the debit logic locally: read the chip, check the balance, decrement, write back, and queue the transaction for sync. The cloud server is not in the critical path of an offline debit. The state that matters for spending sits on the chip itself, so the edge node (the phone) only has to keep the local sync queue, not a shadow copy of the balance. The second level is the server deployment itself. We deploy on Railway, which lets us pick the region where the container and the database run, so the backend sits close to the festivals it serves. For the operations that do need the server (top-ups, sync, admin dashboard), this makes a noticeable difference in responsiveness. Spinning up another region is just a configuration change.

The CAP theorem (Section 2.3.1) told us what we already knew intuitively: during a network partition at a festival, we have to choose between consistency and availability, and refusing transactions because the Wi-Fi dropped is not acceptable. The hybrid model sidesteps the hardest part of this tradeoff by giving the chip authority over the debit lane. The chip is a single, physically consistent copy of the spendable balance that moves with the user, so during an offline period there is no merge problem because there is nothing else to merge with. The CAP tradeoff still applies to the synchronization of transaction logs with the server, but there the worst

case is delayed reporting and bounded loss, not double-spending.

So to sum it up: from the existing systems we took the chip-as-debit-authority idea (transit cards), spending limits as a safety net (EMV), and the lesson that on-site infrastructure should be avoided (festival systems). From the architectural patterns, we took event sourcing for the transaction log and edge computing as the general execution model, and we left behind local-first's strong local ownership model and CRDTs because the lane separation in the hybrid model makes them unnecessary.

3.4 NFC wristbands and offline support

Since the balance lives on the wristband, the offline story becomes much simpler than it would be with a server-side model. This section explains how the wristband interaction works, what happens during offline periods, and how the system reconciles everything once connectivity comes back. We also touch on the security considerations here, because with the balance stored physically on the chip, security is part of the architecture, not something you bolt on afterwards.

3.4.1 How a payment works

The transaction flow between the vendor device and the wristband goes like this:

1. The vendor enters the amount (or selects items from a product catalog) on their phone.
2. The customer taps their wristband against the phone's NFC reader.
3. The app reads the current balance and transaction counter from the wristband.
4. The app checks that the balance is sufficient.
5. The app writes the new (lower) balance and an incremented counter back to the wristband.
6. Only after a successful write confirmation, the transaction is recorded as completed in the local log.
7. If the write fails (customer pulled their wrist away too quickly, interference, etc.), the transaction is marked as failed and the vendor is prompted to retry.

This ordering is important. The vendor device never records a successful payment unless the wristband was actually updated. The worst case scenario is that the vendor has to ask the customer to tap again, which is a small inconvenience but not a financial problem.

3.4.2 Offline operation

During normal online operation, vendor terminals send each transaction to the server right after it is recorded locally. The server updates its own ledger, which is the authoritative record for reporting, settlement, and auditing.

When the network goes down, vendor terminals keep processing payments exactly as before. The wristband interaction does not change at all. The only difference is that transactions accumulate in the local queue on the vendor device instead of being sent to the server. Once connectivity returns, the device pushes its queued transactions to the server in a batch.

The full mechanics of how queued debits and pending credits converge with the server's ledger are covered in Section 3.2. From the point of view of the offline flow, what matters is that the chip's atomic decrement is authoritative, and the server picks up the queued debit as soon as the terminal can reach it.

3.4.3 Top-ups: the one thing that needs the network

Topping up is the one operation that cannot happen offline. When an attendee loads tokens onto their wristband, real money needs to move from their bank account through Stripe and into the event operator's account. This requires an internet connection. Restricting top-ups to online-only is also what keeps the credit lane fully under server control, which is the foundation of the reconciliation model described above.

Top-ups can happen in two ways. First, attendees can top up through the app before or during the event, using Stripe. Once the payment clears, the server records the new balance and increments `credit_counter`. The attendee does not need to do anything special: the next time their wristband is read by any vendor terminal (online), the pending credit is picked up automatically and written to the chip as part of that interaction. Second, dedicated cash top-up stations at the event take cash and submit the credit through the server, which then writes the new balance to the wristband on the spot. In both cases, the server is the only writer of credits, and the chip only ever receives credits via an authenticated server response.

3.4.4 Security of the wristband balance

Storing the balance on the wristband opens up questions that a server-side model does not have. If the balance is just a number written to an NFC chip, what stops someone from using their phone to give themselves more tokens?

For this thesis demo we use a plain NTAG-class NFC chip. The hybrid reconciliation model from Section 3.2 does not depend on chip-side cryptography to

be correct: the counters and the lane separation give the system its convergence properties regardless of whether the chip is authenticated. The price of using normal NFC is that the chip cannot defend itself against a determined attacker who reads, modifies, or clones the data files. Cheap MIFARE Classic chips, as Garcia et al. [GdKGM+08] demonstrated, are particularly easy to break, and plain NTAG chips do not even claim to be secure. Mitigations like `debit_counter` chain checks, server-side anomaly detection, and bounded offline spend limits raise the bar but do not remove the threat.

The full threat model, the upgrade path to authenticated chips like DESFire EV2/EV3 with AES-128 mutual authentication, the key management strategy, and the procedure for handling a detected compromise are all covered in Chapter 5.

3.5 The mobile application

There is one mobile app, built in React Native, that attendees and vendors both download. What the user sees depends on their role: attendees see their balance, and vendors get a POS terminal. The reason we went with React Native is that the same codebase needs to run on both regular phones (for attendees checking their balance) and on the devices that vendors use as POS terminals. We did not want to maintain two separate apps, and React Native gives us one codebase that compiles to both iOS and Android. Since the vendor POS is just a phone with NFC capabilities, the same app handles both use cases. The admin dashboard is a separate React web application, which we cover in Section 3.6.

3.5.1 Attendee view

Attendees see a simple screen showing their current token balance, a transaction history list, and a way to top up their wallet online using Stripe. The balance displayed in the app is synced from the server and shows the last known state. However, the real balance during the event lives on the wristband, not in the app. The app is mostly informational for attendees: it is nice to check your balance and see what you spent, but the actual payment happens by tapping the wristband at a vendor terminal.

3.5.2 Vendor POS mode

When an admin has whitelisted a device from the admin dashboard, the app switches to POS mode on that device. The vendor sees a payment screen where they enter the amount (or select items from a product catalog), and then the customer taps

their wristband against the phone's NFC reader. The app reads the balance, checks that there are enough tokens, deducts the amount, writes the new balance back, and logs the transaction locally. If the device is online, the transaction is also sent to the server immediately. If not, it goes into the local queue and gets synced later.

To become a vendor, the business needs to provide basic information: business name, a contact person, and the type of products they sell. The admin reviews this and can approve or reject the application. Once approved, the admin whitelists specific devices (identified by their device ID) as POS terminals for that vendor. This whitelisting step matters because it means not just any phone can start accepting payments. Only devices that the admin has explicitly authorized will have the POS functionality enabled.

3.5.3 Device security checks

Since the vendor's phone is effectively a payment terminal, the app needs to check some basic security measures before allowing POS mode. A rooted or jailbroken phone is a risk because someone could modify the app to skip the balance check or tamper with the transaction logs. The app checks for root/jailbreak status at startup and refuses to enter POS mode on compromised devices.

Beyond root detection, the app also verifies that it is running the official, unmodified version (not a repackaged APK), that the device has a screen lock enabled, and that the OS version is not critically outdated. These are not bulletproof checks (determined attackers can bypass root detection), but they raise the bar enough to prevent casual tampering and give the system a way to flag suspicious devices to the admin.

3.6 Admin dashboard

The admin dashboard is a separate web application built in React, not part of the mobile app. We made it a website so that any event organizer can access it from any device with a browser, without needing to download anything. This also means it works on laptops, tablets, or phones, whatever the admin has on hand. The dashboard covers four main areas:

Event setup. The admin creates the event, sets the token-to-currency exchange rate, defines spending limits, configures the commission rates for vendors, and sets the event dates. The event goes through a lifecycle (setup, active, settlement, closed) and the dashboard controls the transitions between phases.

Vendor and device management. The admin reviews vendor applications, approves or rejects them, and whitelists specific devices as POS terminals. Each ven-

dor can have multiple devices, and the admin can see which devices are currently active, when they last synced, and how many transactions they have processed.

Monitoring and statistics. During the event, the dashboard shows real-time transaction volume (when devices are online), total tokens in circulation, revenue by vendor, and a timeline of sync events. This gives the organizer visibility into how the event economy is performing.

Security monitoring. The dashboard flags suspicious activity: devices that have not synced in a long time, wristbands with transaction chain breaks (potential tampering), devices that failed root detection checks, and any balance discrepancies flagged during reconciliation. The admin can block a device or wristband directly from the dashboard if something looks wrong.

3.7 Backend server

The backend is built with NestJS, a Node.js framework that gives us a structured way to organize the codebase into modules. We chose NestJS because it enforces a clean separation between controllers (handling HTTP requests), services (business logic), and repositories (database access), which keeps the code organized as the project grows. It also has built-in support for things like guards (for authentication), interceptors (for logging and error handling), and dependency injection, which would otherwise require setting up manually with a bare Express or Fastify server.

The server is deployed as a Docker container on Railway, which gives us a managed PostgreSQL database in the same region as the container. We deploy to a European region so that the latency between the vendor devices and the server stays low for festivals in Romania and the rest of Europe.

The server exposes a REST API that covers four main areas: authentication and accounts (registration, login, token refresh, role-based access control), event management (creating and configuring events, managing the event lifecycle), vendor management (approving vendors, whitelisting devices, setting commissions), and sync/reconciliation (endpoints for vendor devices to push their offline transaction logs and pull updated state).

3.7.1 Why a single service, not microservices

It would be tempting to split the backend into separate services for payments, event management, and synchronization. But for our use case, splitting introduces problems without solving any real ones.

If the payment service needs to check whether a vendor is authorized for an event before processing a sync, and that information lives in a different service, we now have distributed transactions or eventual consistency between our own services. That is exactly the kind of complexity we are trying to avoid. With a single service, a database transaction can atomically check the vendor status and record the payment in one go.

The practical argument is also about operations. Nobody wants to manage five services, five deployment pipelines, and five sets of logs for what is fundamentally one product. With a single service, the entire platform runs as one container image plus a PostgreSQL database. Deploying a new version means rolling out one image. Debugging a production issue means looking at one set of logs. This simplicity is not just convenient, it is what makes it realistic for a small team (or one person, in the case of this thesis) to actually run the thing.

This does not mean the code is a mess internally. NestJS modules give us the same logical separation that microservices would, just without the network boundary between them. The payment module, the event module, and the sync module each have their own services, controllers, and DTOs. If someday the platform grows to the point where splitting makes sense, the module boundaries are already there.

3.7.2 Multi-tenancy

The server handles all festivals from a single deployment. Every table in the database is scoped by an `event_id` column, which acts as the tenant boundary. When a vendor device makes an API call, the request includes the event context (through a path parameter or a token claim), and all queries are filtered by that event. Data from Festival A is never visible to users of Festival B, even though it all lives in the same database.

This is simpler and cheaper than deploying a separate server and database for each festival. With per-festival deployments, the operator would need to provision infrastructure before each event, manage dozens of database instances, handle migrations across all of them, and tear everything down afterwards. With multi-tenancy, adding a new festival is just creating a row in the events table.

3.7.3 Horizontal scalability

Keeping the server as a single service does not mean it is limited to a single instance. The server is designed to be stateless: it stores no session data or cached state in memory. Every request gets everything it needs from the database and from the JWT token in the request header. This means we can run as many instances of the server as we want behind a load balancer, and any instance can handle any request.

There are no sticky sessions, no in-memory caches to synchronize, and no inter-instance communication.

For most festivals, a single instance will be more than enough. The majority of transactions during the event happen on the wristband itself without touching the server at all. The server mostly handles top-ups (which are less frequent), sync batches from vendor devices (which are periodic, not per-transaction), and admin dashboard queries. But if the platform grows to the point where dozens of festivals are running simultaneously and the sync traffic gets heavy, scaling horizontally is just a matter of increasing the replica count.

3.8 Database: PostgreSQL

The database is PostgreSQL, and we are running a single instance of it (with the option to add read replicas later if needed). The schema is intentionally simple: users, events, wallets (one per user per event), transactions, and a sync log that tracks which device uploaded what and when.

3.8.1 Why PostgreSQL

PostgreSQL gives us ACID transactions for the financial data and is mature enough that we do not have to worry about edge cases in the transaction isolation. For a payment system, this matters. When the server processes a top-up, it needs to atomically check the Stripe payment status, create the transaction record, and update the wallet balance. If any step fails, everything rolls back. With PostgreSQL, this is just a regular database transaction.

3.8.2 Why a single database is enough

A common worry with using one database for everything is whether it can handle the load. But the reality is that PostgreSQL can handle a lot more than people give it credit for. The best example is probably OpenAI. In January 2026, they published a blog post [Ope26] describing how they run ChatGPT, a product with over 800 million users, on PostgreSQL. Their setup is a single primary PostgreSQL instance on Azure that handles all writes, with around 50 read replicas spread across multiple regions for reads. The whole thing processes over a million queries per second while keeping p99 latency in the low double-digit milliseconds and maintaining five-nines availability. They did not shard the database or switch to some exotic distributed database. They scaled PostgreSQL with read replicas, connection pooling, query

optimization, and by offloading write-heavy workloads to separate sharded systems when needed.

If that setup can serve 800 million ChatGPT users, a festival payment system with 50,000 attendees is not going to be the thing that breaks PostgreSQL. And in our case, the database load is even lighter than a typical web application. Most payment transactions happen entirely on the wristband and never touch the database in real time. The database mainly handles top-ups, periodic sync batches from vendor devices, and dashboard queries. Even during a large festival, the peak database load is modest because the actual payment processing is offloaded to the wristband.

If the platform does grow to the point where the database becomes the bottleneck, we can follow the same playbook that OpenAI used: add read replicas for distributing query load, use connection pooling (PgBouncer) to handle many concurrent connections efficiently, and partition tables to keep query performance stable as the data grows. These are all standard operational techniques that do not require changing the application code.

Chapter 4

Implementation

This chapter describes how the architecture from Chapter 3 was turned into running code. The focus is on the parts that are specific to the hybrid balance model: the database schema rework, the on-chip data layout, the sync endpoint, and the reconciliation logic that ties everything together. Topics like screen design, navigation, or generic CRUD endpoints are mentioned only briefly because they do not say much about the system as a whole.

4.1 Implementation overview

The system is built on three codebases that live in the same monorepo. The backend is a NestJS server in `backend/`, talking to a Postgres database through Drizzle ORM. The vendor terminal is a React Native application built with Expo, in `softpos/`, using `react-native-nfc-manager` for chip reads and writes. The thesis itself lives in `docs/`.

The whole stack uses TypeScript end to end. This was a deliberate choice. The data shapes that travel between the chip, the terminal, and the server are easy to get wrong, and a single set of types shared between client and server catches most of those mistakes at build time instead of in a tent at three in the morning. Drizzle's schema definitions also act as the source of truth for the database, so the table layout and the application types stay in sync without a separate code generation step.

Postgres was picked for the database because the workload is mostly small writes against a single bracelet row at a time, with occasional reporting queries on top. Serializable isolation, which the reconciliation algorithm relies on, is well supported and behaves predictably. The terminal stores its offline queue in AsyncStorage, which is enough for the volumes a single vendor handles between sync windows.

4.2 Database schema for the hybrid model

The hybrid model from Chapter 3 required a real cleanup of the schema. The original design had a generic `wallets` table where each user owned a single balance, shared across events. That made sense when the system was thought of as a closed-loop wallet platform, but it does not match how festivals actually work. An attendee at one festival does not expect leftover credit to appear at a different festival run by a different organizer, and an organizer wants their event balances to be isolated for accounting and refunds. Balance is therefore event-scoped, and the natural place for it is the row that already represents the link between an attendee and a specific event.

That row is `event_bracelets`, and it now carries the balance directly. Three new columns were added: `balance` (an integer count of cents, defaulting to zero), `debit_counter_seen` (the highest `debit_counter` value the server has applied from the chip for this bracelet), and `credit_counter` (a server-owned counter that increases by one every time the server credits the bracelet, for example after a successful top-up). The two counters are independent on purpose. Debits are allocated by the chip when a payment happens at a vendor, and credits are allocated by the server when money flows in. Keeping them separate means neither side has to coordinate with the other to pick a fresh value.

The `transactions` table was reshaped to match. The legacy `transaction_counter` column was dropped because it tried to merge debits and credits into one sequence, which made it impossible to tell which side a row was authored by. In its place, two nullable columns were added: `debit_counter`, set on debit rows, and `credit_counter`, set on credit rows. Exactly one of the two is populated for any given transaction. The foreign key `wallet_id` was retargeted to `event_bracelet_id`, and the redundant `event_id` column was removed because the event is reachable through the bracelet. The same retargeting was applied to `payment_intents.wallet_id`.

Two tables were dropped entirely. The `wallets` table is no longer needed because the balance moved to `event_bracelets`, and `vendor_invitations` was removed because the invitation flow was replaced by a simpler model where adding a user as a vendor creates the user account on the spot if they do not already have one. The remaining membership tables (`event_tickets`, `event_members`, `vendor_members`) were kept as is.

The end result is a schema that maps cleanly onto how operators think about a festival. A bracelet belongs to an event, the balance lives on that bracelet, and every movement of money against it is logged in `transactions` with a counter that says whether it was a debit (allocated by the chip) or a credit (allocated by the server).

Table	Column	Purpose
event_bracelets	balance	Current balance in cents, event scoped
event_bracelets	debit_counter_seen	Last debit counter applied from chip
event_bracelets	credit_counter	Server-owned credit counter
transactions	debit_counter	Set on debit rows, nullable
transactions	credit_counter	Set on credit rows, nullable
transactions	event_bracelet_id	Replaces wallet_id
payment_intents	event_bracelet_id	Replaces wallet_id

Table 4.1: New and changed columns introduced by the hybrid model rework.

4.3 The reconciliation service and sync endpoint

The sync endpoint is the single place where the chip state, the terminal's offline queue, and the server ledger meet. It is exposed by the backend as a single POST per bracelet, and it is intentionally chunky rather than chatty: the terminal sends everything it has for one bracelet, and the server returns everything the terminal needs to write back to the chip and to clean up its local queue.

```
POST /api/v1/events/:eventId/wristbands/:wristbandUid/sync
Authorization: Bearer <token>
Content-Type: application/json

Request:
{
  "chipState": {
    "balance": <int cents>,
    "debit_counter": <int>,
    "credit_counter_seen": <int>
  },
  "pendingDebits": [
    {
      "idempotencyKey": "<uuid>",
      "amount": <int cents>,
      "vendorId": "<uuid>",
      "counterValue": <int>,
      "deviceId": "<string>",
      "clientTimestamp": "<ISO 8601>"
    }
  ]
}
```

```
Response 200:
{
  "serverState": {
    "balance": <int cents>,
    "debit_counter_seen": <int>,
    "credit_counter": <int>
  },
  "applied":  ["<idempotencyKey>", ...],
  "rejected": [
    { "idempotencyKey": "...",
      "reason": "duplicate" | "insufficient_funds"
              | "counter_gap" | "invalid" }
  ],
  "chipShouldWrite": {
    "balance": <int cents>,
    "credit_counter": <int>
  }
}
```

The terminal does not send the chip's HMAC, because by the time `pendingDebits` reaches the server those debits have already been written to the chip and the HMAC has been recomputed. What matters at sync time is whether each entry in the queue has actually landed on the server's ledger.

The reconciliation algorithm runs inside a serializable Postgres transaction that locks the bracelet row for the duration of the call. Serializable isolation is important here: two terminals that both held the same bracelet during overlapping offline windows can hit the sync endpoint at the same time, and the database needs to make their effects look like a strict ordering. With a weaker isolation level, a race between two reconciliations could let the same chip counter be applied twice, or could allow the balance to go negative.

```
1. Open a serializable transaction and lock the bracelet row.
2. Sort pendingDebits by counterValue ascending.
3. For each pending debit:
     a. If idempotencyKey already exists in transactions,
        record it as duplicate and continue.
     b. If counterValue <= bracelet.debit_counter_seen,
        record it as duplicate and continue.
     c. If bracelet.balance < amount,
        record it as insufficient_funds and continue.
```

```
     d. Decrement bracelet.balance by amount.
     e. Set bracelet.debit_counter_seen = counterValue.
     f. Insert a transaction row with debit_counter
        = counterValue and the supplied idempotencyKey.
4. Read the current bracelet.balance and bracelet.credit_counter.
5. Build chipShouldWrite from that current state.
6. Commit the transaction.
7. Return serverState, applied, rejected, and chipShouldWrite.
```

The four rejection reasons cover the cases that can plausibly happen in the field. The `duplicate` reason is returned when the same debit reaches the server twice, either because the terminal retried after a flaky response or because two terminals scanned the same chip at different points and reported the same counter. Idempotency on `idempotencyKey` catches the first case, the counter check catches the second. The `insufficient_funds` reason fires when the chip view of the world disagrees with the server view, usually after a top-up that has not yet been written back to the chip; the debit was authorized offline based on a stale chip balance, and on reconciliation the server cannot honour it. The `counter_gap` reason is reserved for the case where the server expected a contiguous run of debit counters and the queue has a hole, which suggests a debit went missing or was tampered with. The `invalid` reason is the catch-all for malformed entries.

The response carries `chipShouldWrite`, which is the authoritative state the terminal must push to the chip before completing the sync. This usually only changes the chip if a top-up happened in the meantime, but it is computed and returned unconditionally so the terminal logic does not have to branch on whether a credit occurred.

4.4 Chip data layout and HMAC

The chips used for the bracelets are NTAG class tags, which are cheap and widely available, but do not have the cryptographic features of DESFire. They cannot do mutual authentication on their own, so the security has to come from a signed record written into a single user-memory page. The on-chip layout is small and fixed.

```
balance              uint32 little-endian (cents)
debit_counter        uint32 little-endian
credit_counter_seen  uint32 little-endian
hmac                 bytes(16)  HMAC-SHA256 truncated to 128 bits
                                over the 12 bytes above
```

The first twelve bytes are the state. The last sixteen bytes are an HMAC computed over those twelve bytes using a per-bracelet symmetric key. Anyone reading the chip can read the state, but only someone who knows the per-bracelet key can produce a record that the terminal will accept.

The per-bracelet key is derived rather than stored. A single master key lives behind the backend, and every bracelet's working key is computed as

```
key = HKDF(masterKey, salt = wristbandUid, info = "bracelet-v1")
```

The wristband's UID is read from the chip's read-only manufacturer area, so it cannot be spoofed by simply rewriting the user memory. The `info` string `"bracelet-v1"` fences the derivation against reuse if a future version of the layout ever needs to coexist with this one. In production the master key never leaves the festival HSM. For the demo it is provided to the backend through an environment variable, `BRACELET_SIGNING_KE` which keeps the development setup simple while leaving room for the HSM-backed deployment to drop in without touching application code.

The threat model around plain NFC, including replay of an old signed record after a top-up and physical cloning of the chip, is discussed in Chapter 5. The implementation choices here (the per-bracelet derived key, the inclusion of both counters inside the signed record, and the server-owned `credit_counter`) are all in service of the mitigations described there.

4.5 Online debit path

When a vendor terminal has connectivity, the simplest and safest thing to do is to push the debit to the server first and let the chip catch up afterwards. The terminal posts the debit to a charge endpoint that takes the bracelet, the amount, the vendor identity, and an idempotency key generated on the device. The server opens a serializable transaction on the bracelet row, checks that the balance covers the amount, decrements the balance, allocates the next `debit_counter` value (one above whatever was last applied), inserts a `transactions` row with that counter, and returns the new `balance` and `debit_counter` to the terminal.

The terminal then writes the new state back to the chip in a single NFC session, recomputing the HMAC over the new balance, debit counter, and the unchanged `credit_counter_seen`. If that write succeeds, the transaction is closed. If it fails because the customer pulled their wrist away too early, the server-side debit has already happened and the chip is now stale relative to the ledger. The next time the chip is read by any terminal, the server's view will be lower than the chip's view, and on the next sync the terminal can write the corrected state down. This is the same self-healing path the offline flow relies on, just hit from a different angle.

The decision to do server-first online and chip-first offline is intentional. When the network is up, the server is the cheapest place to enforce invariants and the chip is the more failure-prone half. When the network is down, the chip is the only place where the invariant can be enforced at all. The model swaps which side leads based on what is actually available.

4.6 Offline debit path and the queue

The offline path starts on the terminal. When connectivity is missing, or when the request to the charge endpoint times out, the terminal falls back to local authorisation. It reads the chip, verifies the HMAC against the per-bracelet key (which the terminal can derive itself, because it knows the master key for the duration of the event), and checks that the balance covers the amount.

If the check passes, the terminal performs the debit on the chip atomically inside a single NFC session. The new balance is computed (`balance - amount`), the debit counter is incremented by one, the credit counter seen is left untouched, the HMAC is recomputed over the three new fields, and all twenty-eight bytes are written back to the chip in one shot. The single-session write is important: if the customer pulls away halfway through, the chip is either fully updated or fully unchanged, never half written, because NTAG page writes are atomic at the page level and the layout was sized to fit cleanly into that constraint.

Only once the chip write is acknowledged does the terminal enqueue the pending debit in AsyncStorage. The queue entry carries the same fields the sync request will eventually need: a generated `idempotencyKey`, the `amount`, the `vendorId`, the `counterValue` that was just written to the chip, the `deviceId`, and the local timestamp. Storing the entry only after the chip write means the queue can never contain a debit that did not happen; the worst case is a chip update that the terminal forgot, which the next reconciliation will detect through the counter gap.

When connectivity comes back, the terminal posts the queue to the sync endpoint described in Section 4.3. The server replies with which entries it accepted, which it rejected, and what the chip should hold. The terminal removes the accepted entries from AsyncStorage. Rejected entries are kept and surfaced to the operator with the reason; `duplicate` is silently dropped, `insufficient_funds` and `counter_gap` are escalated to a manual review screen because they imply something happened that the automatic logic should not paper over.

4.7 Topup path

Top-ups are the credit side of the system, and they always go through the server because they involve real money moving through Stripe. The terminal (or the attendee's own phone, depending on the flow) creates a Stripe Payment Intent against the backend, which returns the client secret. The user confirms the payment in the Stripe SDK. On success, Stripe sends a `payment_intent.succeeded` webhook to the backend.

The webhook handler verifies the Stripe signature, looks up the corresponding `payment_intents` row by the intent ID, and runs the credit inside a serializable transaction on the target bracelet. The handler increments `bracelet.balance` by the topped-up amount, increments `bracelet.credit_counter` by one, and inserts a `transactions` row with that new `credit_counter` value and a reference to the payment intent. The serializable isolation level matters here for the same reason it does in the debit path: a webhook retry from Stripe and a manual reconciliation could otherwise both try to apply the same credit. Idempotency is enforced both by the unique constraint on the payment intent ID in `transactions` and by the counter check on the chip when the next sync happens.

After the credit is committed, the next time the bracelet meets a terminal, the sync response carries a `chipShouldWrite` that reflects the new balance and the new `credit_counter`. The terminal writes this back to the chip, and the chip's `credit_counter_seen` catches up to the server's `credit_counter`. From that point on, the chip and the server agree, and the bracelet is ready to be debited again, online or offline.

Chapter 5

Security Considerations

A payment system fails differently from other software. A bug in a feed-ranking algorithm wastes a few seconds of attention; a bug in a wallet either gives money away or takes it from people who earned it. Festivals make this harder because the attack surface is wide and the operator is on-site for only a few days. Vendors hand their phones to teenagers, attendees swap wristbands at the gate, and the network drops out for fifteen minutes at a time. The system has to behave correctly under all of that.

This chapter walks through the threat model in the order an attacker would meet it. We start with how users and devices authenticate to the API, then look at how we keep the wallet honest both online and offline, then explain why we are willing to trust a plain NFC chip with offline debits in this deployment, and finally cover Stripe and data protection. The architectural side of the reconciliation model was already covered in Chapter 3. Here we focus on the security properties of that model, on what an attacker can actually do, and on what stops them.

5.1 Authentication and Authorization

The backend uses Better-Auth as the authentication layer, which issues signed session tokens that the mobile app and the admin dashboard send on every request. Internally these tokens follow a JWT-like structure, with a short-lived access token and a longer-lived refresh token. The access token carries the user's identifier and global role, and nothing else of value. Anything beyond identity (which event the user can touch, which vendor they belong to, whether they can issue refunds) is resolved on the server at request time, never trusted from the token.

We deliberately keep the token small. Putting the full permission set into the token would mean revocation is delayed until expiry, which is the opposite of what we want when a vendor's phone is stolen mid-event. The tokens we issue have a short

access lifetime (about fifteen minutes) and the refresh flow checks the user record on every renewal, so disabling an account at the dashboard takes effect within one refresh cycle.

5.1.1 Roles

After the role cleanup that the system underwent during this thesis, there are exactly two layers of role information stored in the database, and a couple of derived roles that only exist at the vendor scope. The two stored layers are:

- `users.role`: a global role with values `super_admin` or `user`. The `super_admin` role is reserved for platform operators (the people running the SaaS itself, not the festival organizers).
- `event_members.role`: a per-event role with values `admin` or `operator`. `admin` can configure the event, approve vendors, run settlements; `operator` can do day-of operations like topping up bracelets at a cash booth and looking up transactions, but cannot change event configuration.

On top of that, vendors have their own membership table `vendor_members.role` with three values: `owner`, `manager`, and `cashier`. The cashier is the person actually pressing buttons at a stand. The manager handles the menu and product list. The owner is the legal contact and the only one who can change banking details.

The full permission set the API recognises is therefore: super admin, event admin, event operator, vendor owner, vendor manager, and vendor cashier, plus the attendee, which is just an authenticated user with no event or vendor membership but an active bracelet assignment in `event_bracelets`.

5.1.2 The ScopeService

Roles by themselves are not enough. A user who is a vendor cashier at festival A must not be able to read transactions at festival B, even if the API endpoint is the same. To enforce this, every guarded controller resolves the caller's scope through a single `ScopeService` on the backend.

The service exposes three methods that cover all the cases we need: `requireEventRole` (caller must be a member of this event with at least the given role), `requireVendorRole` (caller must belong to this vendor with at least the given role), and `requireEventOrVendorAcc` (caller must be either an event member or a member of any vendor at the event, used for read-only sync endpoints). Each of these throws `ForbiddenException` if the check fails, and returns the resolved entity if it succeeds, so the controller can use the row it just authorised against without a second database round trip.

A useful property of this design is that elevation is implicit and ranked, never additive. The role ranks are ordered (`operator < admin`, and `cashier < manager < owner`), and `super_admin` acts as "admin everywhere" by short-circuiting the check at the top of each method. The event organiser (the user listed as `events.organizer_id`) is treated as event admin for their own events, and an event admin is treated as vendor owner for vendors at their event. The role table never duplicates rows; the privilege flows from the ownership graph.

5.2 Wallet Integrity and Double-Spend Prevention

The wallet is the part of the system that holds money, so it is the part attackers care about. The reconciliation model in Chapter 3 explains how chip and server stay in sync; this section focuses on the integrity properties that survive each of the failure modes an attacker can engineer, separated by online and offline mode because the threat model is genuinely different in each.

5.2.1 Online Mode

When the terminal can reach the server, every debit goes through Postgres. Three properties together make double-spending impossible in this mode.

Serializable isolation on the bracelet row. The transaction that debits a wallet runs at `SERIALIZABLE` isolation, with a row-level lock taken on the bracelet's wallet row before the balance is read. Two concurrent debits on the same bracelet cannot interleave: one wins, the other waits, and if the second one would have caused a negative balance it gets rejected at the application level rather than at the database level. We picked `SERIALIZABLE` over the more common `READ COMMITTED` because the cost (occasional retries on conflict) is small at our transaction rate and the guarantee (no anomalies, ever) means we never have to debug a phantom-read in a bank reconciliation report.

Atomic balance updates. The debit, the counter increment, and the transaction insert all happen in the same database transaction. If any step fails, the whole thing rolls back. There is no window where the balance has been deducted but the transaction row is missing, or where the counter has advanced but the balance has not.

Idempotency keys on transactions. Every debit request from a vendor terminal carries a client-generated `idempotency_key` (a UUIDv4 generated before the network call). The server stores this key alongside the transaction row with a unique constraint. If the same request arrives twice (because the network hiccuped between the request and the response, and the terminal retried), the second attempt fails the unique-constraint check, the server returns the result of the first attempt, and the

user is debited once.

5.2.2 Offline Mode

Offline mode is the harder case. The chip is acting as the authority on its own balance, the terminal queues transactions locally, and the server only finds out what happened when the terminal next syncs. Three mechanisms keep this honest.

Monotonic `debit_counter`. The chip's `debit_counter` field increments by exactly one on every debit, in the same atomic write that updates the balance. The server tracks the highest value it has seen as `debit_counter_seen`. When a terminal eventually syncs, the server checks the sequence: every queued transaction must reference a counter that is strictly greater than the previous one and continuous with the chip's last reported value. A counter that goes backward, or skips, is an anomaly. The server flags the bracelet, freezes it, and surfaces it in the admin dashboard for staff to investigate.

Idempotency keys carry across the offline window. The terminal generates the `idempotency_key` the moment it commits the chip-side debit, before any sync attempt. If a queued transaction is replayed (because the terminal crashed and restored from a backup, or because two terminals somehow share a queue), the server's unique constraint catches it on the first sync and the duplicate is silently dropped.

Bounded loss model. Even with all of the above, a terminal that physically dies before draining its queue takes whatever it had with it. The exposure is bounded by sync frequency. Concretely, with a sync interval of 30 seconds, a peak rate of 1 transaction per second per terminal, and an average ticket of 8 EUR, the worst case is roughly $30 \times 1 \times 8 = 240$ EUR per dead terminal. The festival absorbs this loss. We treat it as an operational cost in the same way a physical cash register absorbs the loss when a bag of cash falls out of an armoured van. It is bounded, predictable, and small relative to the gross revenue of the event.

5.2.3 NFC chip authoritativeness for offline debits

If the chip is just a plain NTAG-class NFC tag with no secure element, why are we willing to let it be the source of truth for offline debits at all? The short answer is that this is a closed-loop festival deployment with bounded loss, and the chip does not need to hold up against a nation-state attacker, only against the kind of person who would actually show up at a festival with a rooted phone. The longer answer is below.

Why a non-secure-element chip is operationally trusted

The chip stores four fields that matter: `balance`, `debit_counter`, `credit_counter_seen`, and an `hmac` computed over those fields keyed by a per-bracelet HMAC key. The per-bracelet key is derived from a festival master key and the chip's UID, so each bracelet has its own key, but every terminal that has ever served the festival can derive any bracelet's key on demand. There is no secure element, no challenge-response protocol, no mutual authentication. The chip is dumb storage with an integrity tag.

This is enough for our purposes for three reasons:

1. **The festival is a closed loop.** Tokens can only be redeemed at vendors that the organiser has approved and whose payouts are reconciled at the end of the event. An attacker who magics ten thousand tokens onto their own bracelet still has to spend them at a real vendor, who in turn submits a transaction the server can see. Anomalies show up at settlement time, which is where the festival catches them.

2. **The chip cannot inflate its balance into the negative.** The terminal verifies the HMAC on read and refuses to debit a bracelet whose tag does not match. So while a determined attacker who has extracted a key can write any balance they want, a casual attacker with a phone and a tutorial cannot. The forensic trail exists either way.

3. **The `debit_counter` chain is monotonic and auditable.** Every debit ever done on a bracelet is reflected in the counter. After the event, ops can replay every counter chain across all terminals' uploaded queues and find any gap, duplicate, or rollback. The chip cannot lie about its own history without leaving traces in the chain that the server reconstructs.

Threats with normal NFC and mitigations

There are three realistic attacks against a plain-NFC closed-loop wallet. We address each in turn.

Cloning. The attacker reads a bracelet's UID, balance, debit counter, and HMAC, and writes them onto a second tag they control. They then use both tags in parallel at two different stands, hoping to spend the balance twice before the system notices. The mitigation is the reconciliation pass. As soon as both terminals come online, the server sees two debit chains starting from the same `debit_counter` value, or two chains whose sequences interleave in a way that no real chip could produce. Either the counter goes backward (one of the clones' debit was committed against

an earlier value) or the same idempotency key arrives twice for distinct amounts. The server freezes the bracelet, blocklists the UID, and pushes the blocklist to all terminals on next sync. The attacker's window is bounded by how often vendors come back online, which in practice is seconds to minutes. They get one ride before the system catches up.

Replay. The attacker reads the chip in state S_0, lets the user spend down to S_1, and then writes S_0 back over the chip to undo the debits. This is a more interesting attack than cloning because it uses only the legitimate bracelet, not a duplicate. The mitigation is the same monotonic counter. The next online tap reads the chip's `debit_counter` and finds it lower than the server's `debit_counter_seen`, which is impossible under the protocol's invariants. The bracelet is frozen on the spot and the attendee is sent to staff. The attacker has burned a bracelet to recover a small balance, which is not a profitable trade.

Terminal-key compromise. The attacker roots a vendor phone (theirs or one they bought at a corner shop) and extracts the cached HMAC keys for every bracelet that has ever tapped it. This is the worst realistic threat because once the attacker has a key, they can forge any combination of balance, counter, and HMAC for the bracelets that key covers. The first line of defence is per-bracelet key derivation: a compromised terminal only exposes the keys for bracelets that have actually been served by that terminal, not the master key, so the blast radius is limited to a few thousand bracelets per compromised device rather than the whole event. The second line, in a production deployment, is to keep the festival master key in a hardware-backed keystore (Android StrongBox, iOS Secure Enclave) and only derive per-bracelet keys on demand inside that boundary. For the thesis demo we accept that a compromised terminal exposes the keys it has cached. We track which bracelets have been served by which terminal, and if a terminal is reported lost or rooted, we can blocklist its bracelet set proactively.

The common thread across all three threats is that the chip's guarantees are weak on their own but strong when the server gets to watch the whole timeline. A single bracelet at a single moment can lie. A bracelet that has to stay consistent with itself across the whole event, against a server that checks the chain on every online tap, cannot.

Future hardening

The reconciliation protocol described in Chapter 3 does not depend on the chip type. It works just as well with a plain NTAG-class tag as with a DESFire EV2 or EV3 secure element. For a production deployment, the recommended hardware upgrade is DESFire EV2 or EV3 with mutual AES-128 authentication. The main thing DES-

Fire adds is that the chip itself cryptographically authenticates to the terminal before any read or write happens, which closes the cloning window almost completely. An attacker who wants to clone a DESFire bracelet has to extract the AES key from the chip, which is a hardware attack that needs laboratory equipment and is not realistic in a festival setting. Replay is closed because every read and write is bound to a per-session nonce that the chip generates. Terminal-key compromise is reduced because the terminal only ever holds the diversified key for the current bracelet, not a long-lived cache.

The point of recording this here is that the thesis demo's NTAG-based chip is a deliberate cost choice, not a design constraint. The reconciliation model survives the upgrade unchanged; the chip type is a parameter, and DESFire is the parameter we would pick for the next deployment.

5.3 Offline Trust Model

When the terminal is offline, the question of who we trust gets sharper because the server is no longer in the loop. The trust model rests on three anchors: the terminal, the bracelet, and the chain that links them.

The vendor terminal as a trusted endpoint. The mobile app is built and signed by us, and only signed builds can register against the API. Device registration ties a specific install to a specific vendor at a specific event, recorded server-side as an entry in the device whitelist. A device that is not on the whitelist cannot acquire the bracelet HMAC keys it needs to validate or write to a chip. If a vendor's phone is reported lost, the admin revokes its registration; the next time that device tries to sync, it is rejected, and any queued transactions still on it are flagged for manual review.

Consumer identity bound to the bracelet. The attendee is identified by the `bracelet_uid` on their chip, plus the active assignment in `event_bracelets` that maps that UID to a user (or to an anonymous wallet, for users who have not registered an account). Identity verification at the chip level happens through the HMAC: the terminal will not engage with a chip whose tag does not verify against a key the terminal can derive. The server-side check, on the next online tap, confirms that the assignment is still active and that the bracelet has not been blocklisted.

Replay prevention through the chain. The `debit_counter` and the per-transaction `idempotency_key` together prevent replay. The counter ensures the chip's history is strictly monotonic; the idempotency key ensures that even if a terminal somehow re-uploads a queued transaction (after a crash, a restore, a merge), the server processes it exactly once. Together they make "the same money got spent twice" an impossible event in normal operation, and a detectable event in the anomalous one.

5.4 Stripe and Payment Security

Real money enters the system through Stripe, and the security boundary there is the cleanest line in the whole architecture. We never see card numbers, CVVs, or anything else in PCI scope. The mobile app uses Stripe's SDK, which collects the card details directly from the user and sends them to Stripe's servers. What comes back to our backend is a `payment_intent_id` and a status. PCI compliance is inherited from Stripe rather than handled by us, which is the only sensible way to do it for a system this size.

Two server-side concerns remain. First, webhooks. Stripe notifies us when a payment succeeds or fails by POSTing to a webhook endpoint, and an attacker who can guess that endpoint could try to forge a success notification to credit a wallet without paying. Stripe signs every webhook with an HMAC over the body, and our handler rejects any webhook whose signature does not verify against the signing secret stored in the server's environment. Second, idempotency on the credit side. A webhook can be delivered more than once (Stripe retries on transient failures), so the handler is written to be idempotent: it reads the `payment_intent.status` from Stripe rather than trusting the webhook payload, and it checks that the wallet has not already been credited for this payment intent before incrementing the balance. A duplicated webhook produces zero extra credit.

Fraud at the card level is handled by Stripe Radar, which scores transactions for suspicion and lets us configure thresholds for blocking, manual review, or 3D Secure step-up. Radar is not perfect, but it is far better than what we could build on our side, and getting it for free is one of the practical reasons for using Stripe in the first place.

5.5 Data Protection

The system holds two kinds of data with security implications: transaction logs (which are financial records and need to be retained for accounting) and personal data (which is regulated under GDPR and which we minimise).

Encryption in transit. All API traffic is HTTPS with TLS 1.2 or higher. The mobile app uses certificate pinning against our backend's certificate, so a compromised CA cannot man-in-the-middle the connection without us noticing. NFC traffic is not encrypted at the radio level, but the HMAC on the chip's payload prevents tampering, and the actual amounts and balances that pass over the air do not constitute personal data on their own.

Encryption at rest. The Postgres instance runs on Neon, which encrypts the underlying storage with AES-256. Backups are encrypted with the same scheme.

Application-level encryption is used only for fields that are particularly sensitive, like Stripe customer IDs.

GDPR considerations. The system stores the minimum amount of personal data we can get away with. The bracelet itself stores no PII at all: only the UID, the balance, the counters, and the HMAC. The mapping from UID to user is in the database, not on the chip, so a lost bracelet leaks nothing about its owner. Users can request deletion of their account; on deletion, we anonymise their record (replace the email and name with a tombstone value) but retain the transaction history with the anonymised user reference for accounting purposes. This is the standard "right to erasure with legitimate interest" carve-out that financial systems use.

Retention. Transaction records are retained for the seven years required by Romanian tax law for financial documents. Personal data is purged on user deletion request as described above, except where it is part of a transaction record covered by the seven-year retention. Server logs (request logs, error logs) are retained for 90 days and then deleted automatically, which is enough to investigate incidents without becoming a liability of its own.

Chapter 6

Discussion

6.1 Design Trade-offs

6.2 Comparison with Existing Solutions

6.3 Limitations

6.4 Further Improvements

Chapter 7

Conclusions

Bibliography

[CP03] Patrick Y. K. Chau and Simpson Poon. Octopus: An E-cash payment system success story. *Communications of the ACM*, 46(9):129–133, 2003.

[DHJ+07] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's highly available key-value store. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP 2007)*, pages 205–220, 2007.

[eFe16] eFestivals. Download festival scraps on site cashless only payment system. `https://www.efestivals.co.uk/news/16/160301h.shtml`, 2016. Online; accessed 2026.

[Eur09] European Parliament and Council of the European Union. Directive 2009/110/EC on the taking up, pursuit and prudential supervision of the business of electronic money institutions. `https://eur-lex.europa.eu/eli/dir/2009/110/oj/eng`, 2009. Online; accessed 2026.

[Eur15] European Parliament and Council of the European Union. Directive (eu) 2015/2366 on payment services in the internal market (PSD2). `https://eur-lex.europa.eu/eli/dir/2015/2366/oj/eng`, 2015. Online; accessed 2026.

[Fow05] Martin Fowler. Event sourcing. `https://martinfowler.com/eaaDev/EventSourcing.html`, 2005. Online; accessed 2026.

[GdKGM+08] Flavio D. Garcia, Gerhard de Koning Gans, Ruben Muijrers, Peter van Rossum, Roel Verdult, Ronny Wichers Schreur, and Bart Jacobs. Dismantling MIFARE classic. In *Proceedings of the 13th European Symposium on Research in Computer Security (ESORICS 2008)*, volume 5283 of *LNCS*, pages 97–114. Springer, 2008.

[GL02] Seth Gilbert and Nancy Lynch. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, 2002.

[KB17] Martin Kleppmann and Alastair R. Beresford. A conflict-free replicated JSON datatype. Technical Report 10, 2017.

[KWvHM19] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: you own your data, in spite of the cloud. In *Proceedings of ACM Onward! 2019*, pages 154–178, 2019.

[MDAB10] Steven J. Murdoch, Saar Drimer, Ross Anderson, and Mike Bond. Chip and PIN is broken. In *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, pages 433–446, 2010.

[Ope26] OpenAI. Scaling PostgreSQL to power 800 million ChatGPT users. `https://openai.com/index/scaling-postgresql/`, 2026. Online; accessed April 2026.

[Rie11] Eric Ries. *The Lean Startup*. Crown Business, New York, 2011.

[SBH24] Lachlan Schomburgk, Alessandro Belli, and Arvid O. I. Hoffmann. Less cash, more splash? A meta-analysis on the cashless effect. *Journal of Retailing*, 100(3):382–403, 2024.

[SCZ+16] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.

[SDK24] SDK.finance. Closed-loop payment systems: Definition, benefits, and business cases. `https://sdk.finance/blog/closed-loop-payment-systems-definition-benefits-and-business` 2024. Online; accessed 2026.

[Shi02] Yasutomo Shirakawa. JR East contactless IC card automatic fare collection system "Suica". In *Proceedings of the 7th IEEE International Symposium on High Assurance Systems Engineering (HASE 2002)*, pages 3–10, 2002.

[SJL+13] Muhammad Zubair Shafiq, Lusheng Ji, Alex X. Liu, Jeffrey Pang, Shobha Venkataraman, and Jia Wang. A first look at cellular network performance during crowded events. In *Proceedings of the ACM SIGMETRICS 2013*, pages 17–28, 2013.

[Son24a] Sony Corporation. Case study: Hong Kong octopus card. https://www.sony.co.jp/en/Products/felica/casestudy/article/octopus.html, 2024. Online; accessed 2026.

[Son24b] Sony Corporation. FeliCa – contactless IC card technology. https://www.sony.net/Products/felica/about/, 2024. Online; accessed 2026.

[SPBZ11] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, pages 386–400, 2011.

[Tap24] Tappit. RFID cashless payments for events. https://tappit.com/pay/rfid, 2024. Online; accessed 2026.

[Vog09] Werner Vogels. Eventually consistent. *Communications of the ACM*, 52(1):40–44, 2009.

[You10] Greg Young. CQRS documents. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf, 2010. Online; accessed 2026.